

文档编号	TY-SUM-20231030A
版本	A
页数	55 页

模型库和函数库开发规范

科学计算与系统建模仿真项目组

编制	王天飞、张新城、周雯	生效日期	2023-10-30
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2023.03.14	V0.1	初始建立	王天飞	郭俊峰	刘奇
2023.07.19	V0.5	添加模型库资源文件组织规范、 修改模型测试规范	王天飞	郭俊峰	刘奇
2023.10.30	V1.0	修订函数库开发规范	张新城	郭俊峰	刘奇

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

目 录

1. 函数库开发规范	1
1.1 函数库开发规范.....	1
1.1.1 目录结构规范.....	1
1.1.2 函数定义规范.....	3
1.1.3 工程管理规范.....	5
1.1.4 单元测试规范.....	6
1.1.5 Julia 编码规范	7
1.2 外部语言封装规范.....	14
1.2.1 C/C++语言封装规范.....	14
1.2.2 Python 语言封装规范	16
1.3 M 语言接口规范	17
1.3.1 API 列表	18
1.3.2 M 类型与 Julia 类型对应关系.....	25
1.3.3 Julia 类型与 M 类型对应关系.....	25
2. 模型库开发规范	26
2.1 模型库开发规范.....	26
2.1.1 模型库组织结构规范.....	26
2.1.2 模型编码规范.....	28
2.1.3 模型图形布局规范.....	37
2.1.4 模型用户指南编写规范.....	37
2.1.5 模型工程化实践规范.....	38
2.1.6 模型测试规范.....	40
2.1.7 模型库版本管理规范.....	43
2.2 外部语言封装规范.....	44
2.2.1 C/C++语言封装规范.....	44
2.2.2 Julia 函数库集成规范	46

1. 函数库开发规范

科学计算环境内置了数学、图形、图像、符号数学、曲线拟合、信号处理、通信、DSP 系统、控制系统、优化、统计等多维度的函数库，同时也允许用户开发新的函数库并集成到科学计算环境中，从而扩展科学计算环境功能。

函数库是指存放可重用功能代码的项目，主要以 Julia 语言实现。函数库是一个文件夹，其中包含了一系列的 Julia 源代码和配置文件。

本规范包含开发函数库的相关约束和标准，包含三个部分：函数库的开发规范、外部库的封装规范、M 语言接口规范。

1.1 函数库开发规范

函数库的开发过程中需要遵循相应的开发规范，以确保函数库的高质量和可扩展性，并方便其在 Syslab 环境中进行集成和管理。同时为了保证 Julia 函数库的可维护性，开发函数库需要遵守 Julia 编码规范。

1.1.1 目录结构规范

一个标准的包（如 Revise），其目录结构如下图所示：

- docs: 帮助文档文件夹
- examples: 可选，测例文件夹
- images: 可选，资源文件夹
- src: 库源码文件夹
- test: 单元测试文件夹
- Project.toml: 项目文件
- LICENSE.md: 许可证文件
- README.md: 库说明文件
- ...: 用户还可以添加其它文件夹

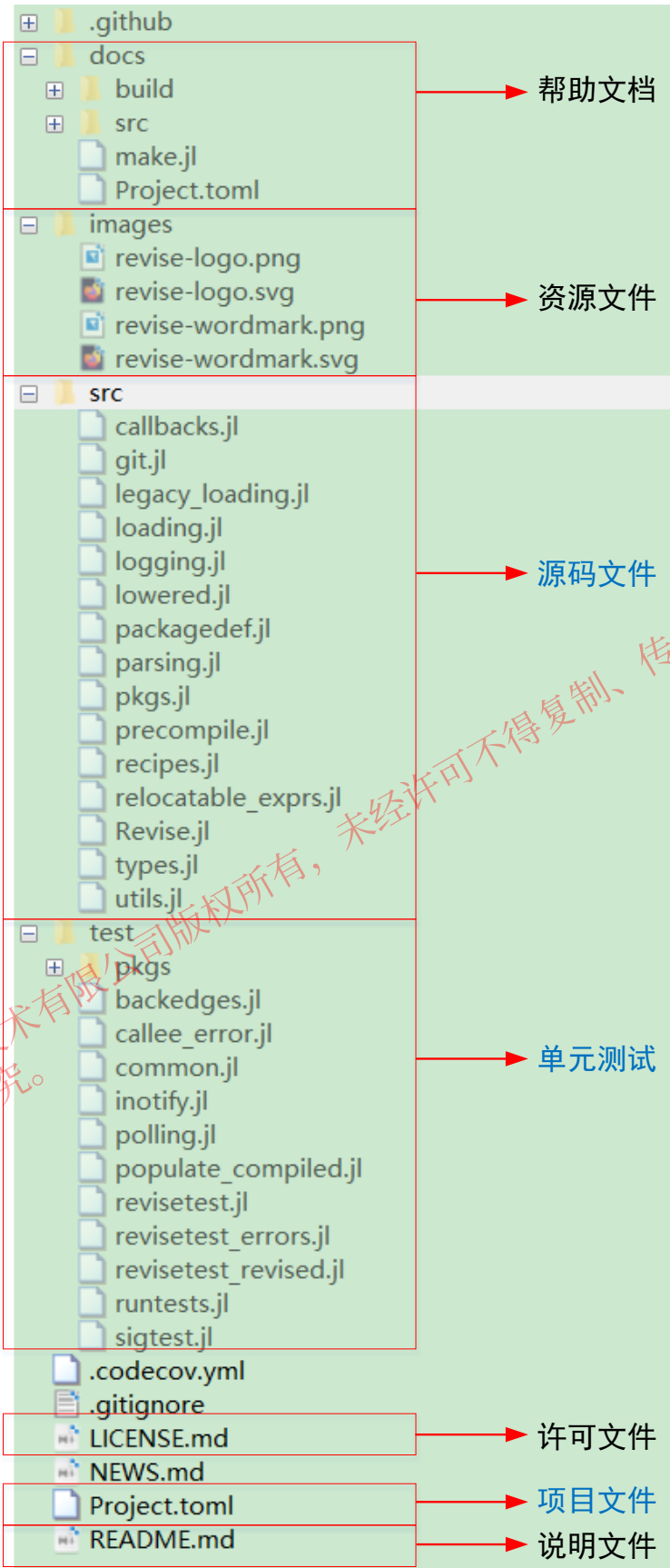


图 1-1 函数库的目录结构

1.1.2 函数定义规范

1.1.2.1 导出列表

函数、类型、全局变量和常量等，可以通过 `export` 添加到模块的导出列表。通常，导出接口列表位于或靠近模块定义的顶部，以便读者可以轻松找到它们。

首先，可以看下典型 Julia 包的做法，如下图所示：

```
1 if !isdefined(Base, :Experimental) && !isdefined(Base.Experimental, Symbol("@optlevel"))
2   @eval Base.Experimental.@optlevel 1
3 end
4
5 using FileWatching, REPL, Distributed, UUIDs
6 import LibGit2
7 using Base: PkgId
8 using Base.Meta: isexpr
9 using Core: CodeInfo
10
11 export revise, includet, entr, MethodSummary
```

图 1-2 Revise 库的导出列表

例如，MyExample 包的导出列表，定义如下：

```
module MyExample

export greet

greet() = print("Hello World!")

...

end # module
```

1.1.2.2 函数定义

Julia 包中最常用的就是函数，函数定义由两部分组成：一是函数注释，包括函数原型和函数功能说明；二是函数的算法实现。

首先，可以看下典型 Julia 包的做法，如下图所示：

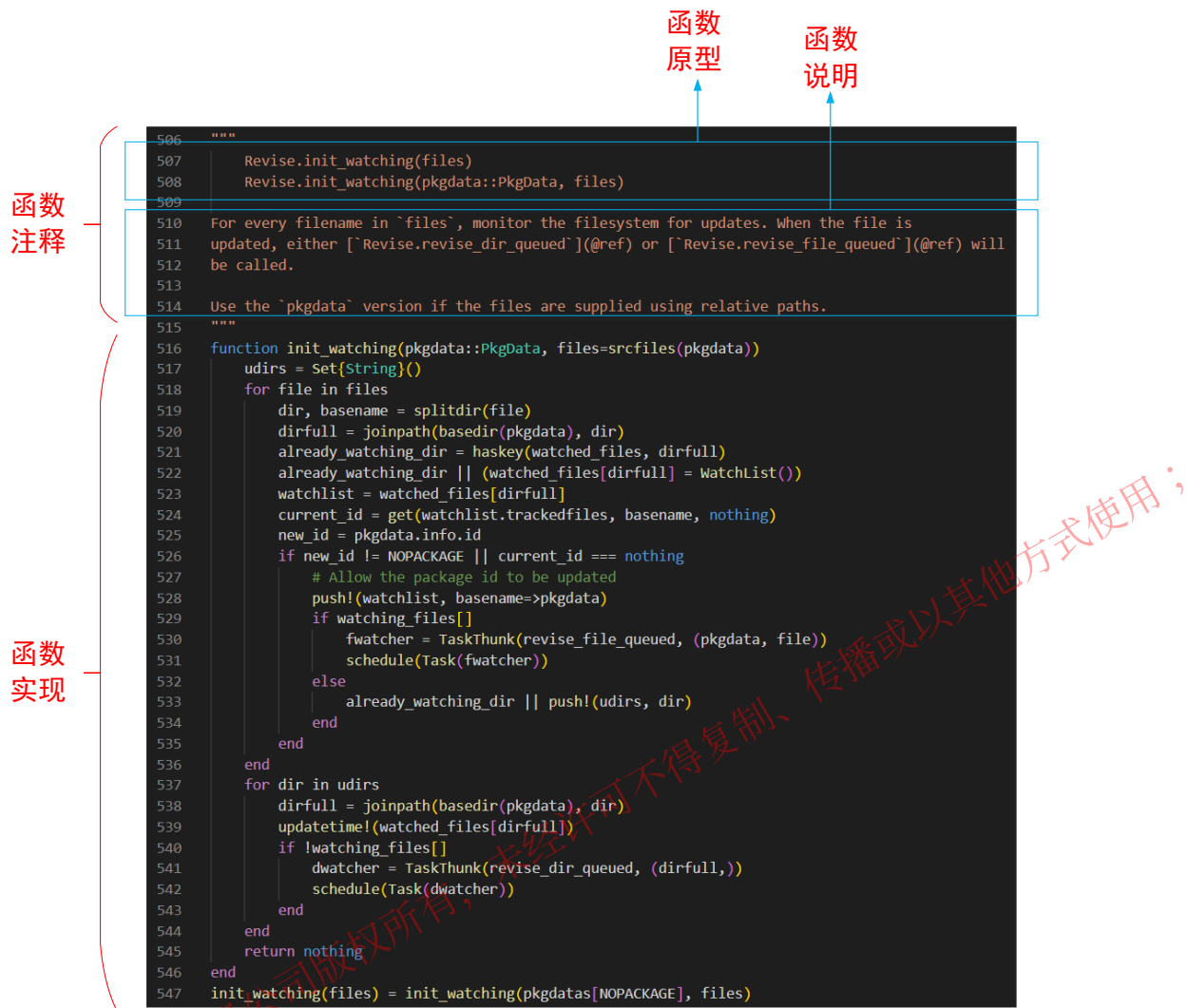


图 1-3 Revise 库的函数定义

例如，为 MyExample 包添加“domath”和“pythagoras”函数，如下所示：

```
module MyExample
export greet, domath, pythagoras

greet() = print("Hello World!")

"""
    domath(x::Number)

Return `x + 5`.
"""
domath(x::Number) = x + 5

include("math.jl")

end # module
```

其中，MyExample/src/math.jl 中的函数定义如下：

```
"""
    pythagoras(a,b)
```

勾股定理，英文名为 **Pythagoras** 也称为毕达哥拉斯定理。

在平面上的一个直角三角形中，两个直角边边长的平方加起来等于斜边长的平方。
如果设直角三角形的两条直角边长度分别是`a`和`b`，斜边长度是`c`，那么数学公式为：

$$c = \sqrt{a^2 + b^2}$$

返回斜边长`c`

"""

```
function pythagoras(a, b)
    c = sqrt(a^2 + b^2)
end
```

1.1.3 工程管理规范

每个函数库包含一个项目文件 **project.toml**，用于对函数库进行工程管理，其中包括函数的信息、函数库的依赖信息等，如下图所示：

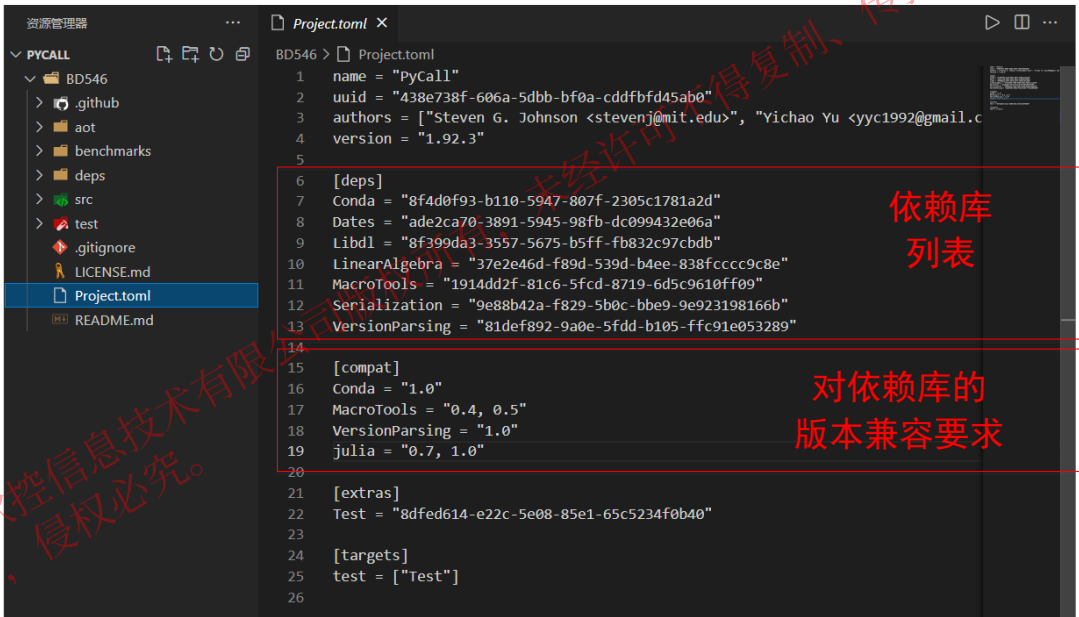


图 1-4 依赖库及其版本兼容要求

包的项目文件 **project.toml**，内容解释如下：

- **name**: 包的名称；
- **uuid**: 包的唯一标识；
- **authors**: 包的作者，书写规则为“[NAME <EMAIL>, NAME <EMAIL>]”；
- **version**: 包的版本。必须遵守 SemVer 语义化版本，简而言之：
 - 破坏性更新：主版本 major release
 - 新特性：小版本 minor release

➤ bug 修复：补丁版本 patch release

- [deps]: 该库依赖的其它函数库，书写规则为“name = uuid”;
- [compat]: 该库对依赖库的版本兼容要求。关于 compat 字段的规则，请参考 <http://pkgdocs.julialang.org/v1/compatibility/>。典型的写法是：

```
[compat]
# 指 D 的所有 0.1.* 和 0.2.* 的版本都兼容
D = "0.1, 0.2"
```

- [extras]: 单元测试规定的依赖库，与“[targets]”一起使用;
- [targets]: 单元测试规定的依赖库。

注，extras 和 targets 请参考 <https://pkgdocs.julialang.org/v1/creating-packages/#Test-specific-dependencies-1>。

1.1.4 单元测试规范

1.1.4.1 单元测试

对于绝大多数代码开发来说，测试是保障代码质量和可靠性的最有力的工具。在这里我们介绍测试的基本内容和手段，以单元测试为主。

Julia 的测试代码存放在“<包根目录>/test”文件夹内并通过“pkg> test”调用“test/runtests.jl”文件。该文件可以理解为 Julia 单元测试的 main 文件。

(1). @test: 检查表达式的结果是否为 true。如果为 true 则测试通过。

```
using Test
@test 1 + 1 == 2
@test ones(2, 2) == [1 1; 1 1]
```

```
julia> using Test
julia> @test 1 + 1 == 2
Test Passed
Expression: 1 + 1 == 2
Evaluated: 2 == 2

julia> @test ones(2, 2) == [1 1; 1 1]
Test Passed
Expression: ones(2, 2) == [1 1; 1 1]
Evaluated: [1.0 1.0; 1.0 1.0] == [1 1; 1 1]
```

图 1-5 @test 的运行结果

(2). @testset: 用于将各个测试组织在一起，例如：

```
@testset "math" begin
    @test 1 + 1 == 2
    @test 1 - 1 == 0
    @test 1 / 0 == 1
end
```

其中，不通过的测试会被标记为 "Fail"，此时要么是测试代码没写对，要么是对应的功能存在 bug。

```

math: Test Failed at REPL[9]:4
  Expression: 1 / 0 == 1
  Evaluated: Inf == 1
Stacktrace:
 [1] macro expansion
   @ C:\Program Files\MWorks.Syslab 2022\Tools\julia-1.7.3\share\julia\stdlib\v1.7\Test\src\Test.jl:445 [inlined]
 [2] macro expansion
   @ REPL[9]:4 [inlined]
 [3] macro expansion
   @ C:\Program Files\MWorks.Syslab 2022\Tools\julia-1.7.3\share\julia\stdlib\v1.7\Test\src\Test.jl:1283 [inlined]
 [4] top-level scope
   @ REPL[9]:2
Test Summary: | Pass  Fail  Total
math          |    2     1     3
ERROR: Some tests did not pass: 2 passed, 1 failed, 0 errored, 0 broken.

```

图 1-6 @testset 的运行结果

1.1.4.2 常用工具

Julia 开发生态最常用的测试工具如下：

- [Test](#)：为 Julia 标准库；
- [ReferenceTests](#)：将结果与参考文件绑定做交叉对比测试，常用于图片的测试；
- [Suppressor](#)：用于抑制或捕获 stdout/stderr 的输出结果，经常与 Test 联合使用；
- Random.seed!：用于重置随机数种子，对于某些不定的情况会有用。

1.1.5 Julia 编码规范

1.1.5.1 语法概要

- 用 4 个空格缩进
- 一行 92 个字符
- 模块名和类型名用大骆驼命名格式
- 方法名用小写字母+下划线的形式(注意区别于官方规则：官方建议尽量不用下划线)
- 用 using 引入模块，每个模块一行，在文件的开头引入
- 写尽可能详细的注释
- 合理利用空格增加代码可读性
- 行尾不要空格
- 避免括号后追加空格：Int64(value) 好过 Int64(value)

1.1.5.2 详细规则

1.1.5.2.1 代码规范

模块引入

- 每次引入一个
- 根据模块名字字母排列
- 引入的函数按照字母或者实际意义顺序排列
- 引入内容很长时, 可以按照合理规则分组
- 推荐用 `using` 而不是用 `import`

二者唯一的区别是, `using` 引入的 `module`, 调用其内部函数时需要加 `module` 名, 而 `import` 引入的则不需要:

```
# Yes:
using Example

Example.hello(x::Monster) = "Aargh! It's a Monster!"
Base.isreal(x::Ghost) = false

# No:
import Base: isreal
import Example: hello

hello(x::Monster) = "Aargh! It's a Monster!"
isreal(x::Ghost) = false
```

函数输出

- 函数输出语句需要放在模块的头部, 在引入依赖以后;
- 不要对一个 `export` 语句换行, 要么一行一个, 要么按需分组;

```
# Yes:
export foo
export bar
export qux

# Yes:
export get_foo, get_bar
export solve_foo, solve_bar

# No:
export foo,
    bar,
    qux
```

全局变量

- 尽量避免使用全局变量;

- 如果需要使用, 加前缀 `const`, 命名规则为全部大写;
- 全局变量在文件的头部位置定义, 紧跟在 `imports` 和 `exports` 之后

函数命名

- 函数名需要能表示函数的属性或者功能, 不需要体现参数类型:
`submit_bid(bid)`应该改成 `submit(bid::Bid)`;
- 函数名尽量控制在一到两个单词内, 小写加下划线分隔;
- 函数越简单约好, 尽量把复杂函数拆分;
- 只用于内部使用的函数, 用下划线开头;

方法定义

- 只有在函数非常短的时候, 才用短函数定义方式

```
# Yes:
foo(x::Int64) = abs(x) + 3
```

```
# No:
foobar(array_data::AbstractArray{T}, item::T) where {T<:Int64} = T *
    abs(x) * abs(item) + 3 for x in array_data
]
```

- 用长函数定义的语法时, 用 `return` 显式返回, 且显式声明返回值:

```
# Yes:
function Foo(x, y)
    # code ...
    return nothing
end
```

- 参数列尽量不要换行, 除非特别长, 如果要换行, 就要每个参数一行, 或者按照位置参数和关键字参数分组换行, 不要随意地到长度限制再换行:

```
# Yes:
function foobar(
    df::DataFrame,
    id::Symbol,
    variable::Symbol,
    value::AbstractString,
    prefix::AbstractString="",
)
    # code
end
```

```
# Ok:
function foobar(df::DataFrame, id::Symbol, variable::Symbol, value::AbstractString,
    prefix::AbstractString="")
    # code
end
```

```
# No: Don't put any args on the same line as the open parenthesis if they won't all fit.
function foobar(df::DataFrame, id::Symbol, variable::Symbol, value::AbstractString,
    prefix::AbstractString="")
```

```
# code
end
```

- 关键字参数需要用;和位置参数区分开

空格

- 紧跟着各种括号内部不要空格: `foo(ham[1],[eggs])`好过 `foo( ham[ 1 ], [ eggs ] )`
- 逗号和分号前面不要加空格;
- 比较和赋值等二元操作符两端要加空格, `=`, `+=`, `==`, `->`等;
- 一些数值相关的二元操作符两侧不加空格, `^`, `//`, `-1`;
- 在 Range 定义中, `:`两侧不要加空格, 复杂的 Range 定义用括号括起来, 避免歧义: `(1+2):(3+4)`
- 不要为了对齐引入超过一个的空格:

```
# Yes:
x = 1
y = 2
long_variable = 3
```

```
# No:
x      = 1
y      = 2
long_variable = 3
```

- 不要引入不必要的空行, 比如同一个函数的不同单行分派之间不需要换行;
- 如果函数调用语句中, 参数过长(比如嵌套函数), 则要按照如下规则换行: 函数的左右括号要在同一缩进水平, 参数要多缩进一级, 参数和关键字在多数情况下应该每个一行。

```
constraint = conic_form!(
    SOCElemConstraint(temp2 + temp3, temp2 - temp3, 2 * temp1),
    unique_conic_forms,
)
```

- 数组或者元组定义的时候, 括号缩进规则类似:

```
arr = [
    some_long_sub_array_A,
    some_long_sub_array_B,
]

nestedarr = [
    [
        some_long_A,
        some_long_B
    ],
    [
        another_long_A,
        another_long_B
    ]
]
```

- 三引号, 三反引号需要缩进:

```
str = """
hello
world!
"""
cmd = `
program
  --flag value
parameter
`
```

- 用空行分隔不同的多行代码块:

```
# Yes:
if foo
  println("Hi")
end

for i in 1:10
  println(i)
end

quote
  x = 1
  y = 2
  a = x + y
end
```

- 在控制流和返回语句之间添加空行:

```
function foo(bar; verbose=false)
  if verbose
    println("baz")
  end

  return bar
end
```

数据结构和控制流

- 具名元组中的=类似于关键字参数的用法, 两端不加空格, 空具名元组应该写成 NamedTuple() 而不是 (;)
- 浮点数不要省略小数点前后的 0: 0.1, 2.0 而不是 .1 2.
- 三元操作符?: 只能用于单行语句, 如果语句超过单行, 则应该用传统的 if elseif else 的方式
- for 循环中应该只用 in 关键词, 尽量不要用 = 或者 ∈, 在列表展开式中也适用

类型标注

- 类型标注需要尽可能的抽象, 而不是尽可能的具体
- 如果需要性能, 则可以用类型参数的方式:

```
# 类型参数
mutable struct MySubString{T<:AbstractString} <: AbstractString
  string::T
  offset::Integer
end
```

```

    endof::Integer
end
# 相比于传统方式:
mutable struct MySubString <: AbstractString
    string::AbstractString
    offset::Int
    endof::Int
end

```

注意，在类型参数的情况下，数据结构在定义的时候就确定了，在整个生命周期中数据类型是限定了的，所以会有更好的性能优化。可以在开发的时候使用下边这种抽象类型，在优化的时候再改成类型参数的模式。

版本声明

为了保持简洁和一致性，声明版本的时候不要使用脱字符^:

```

# Yes:
DataFrames = "0.17"

# No:
DataFrames = "^0.17"

```

代码注释

- 注释是用来解释(复杂的)代码的逻辑的，不要在简单代码上也过度注释
- 注释要随着代码的更新而更新
- 注释应该是完整的语句，首字母大写(除非首字母小写有特殊意义，比如是代码中的指示词/变量)
- 短注释的末尾句号可以省略，块注释需要在每句结尾加上句号
- 如果注释不能保证在与代码同行时不超过单行限制，就不要把注释写在代码同行
- 注释与同行代码之间至少要有两个空格，#与注释内容之间有一个空格
- 提到 Julia 的时候，通常 Julia 指代 Julia 语言，julia 指代 Julia 可执行命令

文档

- 大多数模块、类型、函数都应该有规范的 docstrings(尤其是需要 export 的函数)，但是内嵌函数或者很简单的函数不需要额外添加 docstrings
- 给一个函数添加文档，而不是给函数的一个方法添加文档
- 只有新方法与老方法有区别时，才给已有文档的函数的新方法添加文档
- 文档需要用 Markdown 格式书写，同样遵守 92 chars 的规则
- 复杂参数的方法建议用以下文档模板：

```

"""
mysearch(array::MyArray{T}, val::T; verbose=true) where {T} -> Int

```

Searches the `array` for the `val`. For some reason we don't want to use Julia's builtin search :)

Arguments

- `array::MyArray{T}`: the array to search
- `val::T`: the value to search for

Keywords

- `verbose::Bool=true`: print out progress details

Returns

- `Int`: the index where `val` is located in the `array`

Throws

- `NotFoundError`: I guess we could throw an error if `val` isn't found.

"""

function mysearch(array::AbstractArray{T}, val::T) where T

...
end

1.1.5.2.2 测试规范

● 测试组

Julia 中提供了测试组(test sets)来方便用户按照应用逻辑对测试代码进行分组。runtests.jl 文件最好只包含一个主测试组, 其他分组可以嵌套在主分组内。

● 比较操作

大多数测试会写成@test x == y 的形式, ==不会进行类型检查, 所以@test 1.0 == 1 是可以通过的, 所以不必要为了类型一致牺牲代码可读性:

Yes:

@test value == 0

No:

@test value == 0.0

1.1.5.2.3 性能和优化

Julia 中很多性能提升都是来自于针对输入类型对实际调用函数进行优化。

- 尽量不要声明全局变量(会阻碍 Julia 进行代码优化), 如果要声明, 加上前缀 const
- 第一次实际调用某函数时, 会根据输入类型编译该函数, 所以测试函数性能不应该测试其第一次执行的性能, BenchmarkTools 中的@benchmark 和 @btime 宏可以执行多次函数, 得到更客观的性能测试结果

1.1.5.2.4 其它注意事项

A. 不推荐在函数库中提交 Manifest.toml 文件

Manifest.toml 文件的目的是为了将整个项目的依赖完全锁定在一个具体版本，从而可以通过 `pkg> instantiate` 来得到一个指定的运行版本。对于函数库的开发来说，当你在 Project.toml 中记录了足够完整的 `compat` 字段后，就不太会再需要提供 Manifest.toml 文件了。

B. 禁止使用不带限制的 @reexport 命令

Reexport.jl 提供了一个非常方便的宏命令 `@reexport` 用于将其他 Julia 包所导出的名字再一次导出。但是，在使用时需要注意：

- 禁止使用不带限制的 `@reexport using SomePkg`
- 推荐使用限定符号的 `@reexport using SomePkg: sym1`

以上 Julia 编码规范来源于：[Blue: a Style Guide for Julia](#)，基于这个规范翻译而来并做了修订。

1.2 外部语言封装规范

在 Syslab 平台，我们提供了基于 Julia 的函数库。对于那些使用 C/C++ 或 Python 开发的外部语言函数库，我们可以通过封装成 Julia 函数库来快速集成到 Syslab 平台中。这种封装的方式使得外部语言函数库能够与 Syslab 平台进行无缝集成。通过将外部语言函数库进行封装，我们可以提供统一的接口，使得用户可以直接在 Julia 中调用这些外部语言函数库的功能，而无需关注底层的实现细节和语言差异。

封装外部语言函数库的过程往往涉及到对函数接口的映射和数据类型的转换，以确保外部语言函数库在 Syslab 平台中的正常运行。

1.2.1 C/C++语言封装规范

1.2.1.1 对 C/C++库的要求

为了满足跨平台的要求，对于使用 C/C++ 开发的函数库，我们需要同时提供适用于 Windows 和 Linux 系统的库文件，即动态链接库（DLL）和共享对象库（SO）。

通过提供 DLL 和 SO 库，我们可以确保函数库在不同操作系统下的兼容性。DLL 是 Windows 平台上常见的库文件格式，而 SO 则是 Linux 平台上的标准库文件格式。通过同时提供这两种库文件，可以为不同的操作系统封装相应的函数库。

1.2.1.1.2 ccall 组件

ccall 组件是 Julia 语言内核提供的调用 C/C++ 动态库的模块。通过 ccall 组件可以将 C/C++ 函数封装成 Julia 函数。ccall 函数使用方法如下：

```
ccall((function_name, library), returntype, (argtype1, ...), argvalue1, ...)
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)
ccall(function_pointer, returntype, (argtype1, ...), argvalue1, ...)
```

其中，参数解释如下：

- function_name: 需要调用的 C 函数名
- library: 包含函数的 C 库文件名
- returntype: 函数的返回类型
- argtype: 函数的输入参数类型列表
- argvalue: 函数的输入参数值

1.2.1.1.3 数据类型转换

使用 ccall 函数调用 C++ 库函数时，需要遵循 C/C++ 数据类型与 Julia 数据类型之间的映射规则。目前，基本的 C/C++ 值类型可以转换为 Julia 类型，以 C 为前缀。Julia 数据类型转换为 C/C++ 数据类型的对应关系如下表所示。

表 1-1 Julia 转 C

C 类型	标准 Julia 别名	Julia 基本类型
unsigned char	Cuchar	UInt8
bool (Bool in C99+)	Cuchar	UInt8
short	Cshort	Int16
unsigned short	Cushort	UInt16
int, BOOL (C, typical)	Cint	Int32
unsigned int	Cuint	UInt32
long long	Clonglong	Int64
unsigned long long	Culonglong	UInt64
intmax_t	Cintmax_t	Int64
uintmax_t	Cuintmax_t	UInt64
float	Cfloat	Float32
double	Cdouble	Float64
complex float	ComplexF32	Complex{Float32}
complex double	ComplexF64	Complex{Float64}
ptrdiff_t	Cptrdiff_t	Int
ssize_t	Cssize_t	Int
size_t	Csize_t	UInt

void		Cvoid
void and [[noreturn]] or _Noreturn		Union{}
void*		Ptr{Cvoid} (或 Ref{Cvoid})
T* (where T represents an appropriately defined type)		Ref{T} (只有当 T 是 isbits 类型时, T 才可以安全地转变)
char* (or char[], e.g. a string)		Cstring if NUL-terminated, or Ptr{UInt8} if not
char** (or *char[])		Ptr{Ptr{UInt8}}
jl_value_t* (any Julia Type)		Any
jl_value_t* const* (一个 Julia 值的引用)		Ref{Any} (常量, 因为转变需要写屏障, 不可能正确插入)
va_arg		Not supported
... (variadic function specification)		T... (其中 T 是上述类型之一, 当使用 ccall 函数时)
... (variadic function specification)		; va_arg1::T、va_arg2::S 等 (仅支持@ccall 宏)

1.2.2 Python 语言封装规范

1.2.2.1 PyCall 组件

PyCall 是 Julia 语言调用 Python 库的组件, 这个组件提供了直接调用 Python 库并与 Python 完全互操作的能力。通过 PyCall 组件可以将 Python 函数封装成 Julia 函数。

PyCall 组件可以从 Julia 导入任意的 Python 模块, 调用 Python 函数(在 Julia 和 Python 之间自动转换类型), 从 Julia 方法定义 Python 类, 并在 Julia 和 Python 之间共享大型数据结构。

1.2.2.2 数据类型转换

在使用 PyCall 调用 Python 函数或库时, 需要使用 Python object interfaces 来表示和操作 Python 对象, 这些接口如下表所示

表 1-2 Python object interfaces

Python 对象	功能描述
PyObject	一个表示 Python 对象的 Julia 类型。通常情况下，Python object 的返回值都是 PyObject 类型
PyNone	表示 Python 中的 None 对象
PyCall.PyString	表示 Python 中的字符串对象。可以通过 string 函数将 Julia 的字符串转换为 PyCall.PyString 对象
PyCall.PyDict	表示 Python 中的字典对象。可以通过 Dict 函数将 Julia 字典转换为 PyCall.PyDict 对象，并且可以使用 get, setindex 和 index 等函数进行访问
PyCall.PyTuple	表示 Python 中的元组对象。可以通过 Tuple 函数将 Julia 元组转换为 PyCall.PyTuple 对象，并且可以使用 getindex 和 setindex 等函数进行访问
PyCall.PyArray	表示 Python 中的数组对象。可以通过传递 NumPy 数组到 PyCall.PyArray 函数，得到等效于 python 的 ndarray 的 Julia 对象，PyArray 还支持广播
PyCall.PyAny	表示 Python 中的任何对象。可以在函数的参数列表中使用 PyAny，以便接受任何 Python 对象或类型

这些 Python object interfaces 提供了一些基本的 API，使得 Julia 可以与 Python 交互，并且可以方便地访问和操作 Python 对象。同时，PyCall 还提供了一些额外的函数和工具，可以帮助用户更方便地使用 Python 库。

1.3 M 语言接口规范

TyMLang 是同元 M 语言兼容工具，通过该工具可以实现 M 语言调用 Julia 函数，具体调用过程如下：

- 1.将 M 对象转换成 Julia 对象。
- 2.调用 Julia 函数，用到第 1 步的 Julia 对象，获取到 Julia 类型的结果。
- 3.把 Julia 类型的结果转换成 M 类型结果。

Julia 函数库开发要遵循 M 语言兼容规范，主要包括 Julia 函数的语法、功能定义、函数输入和输出要和 M 语言函数保持一致。

M 语言兼容工具提供了 M 语言调用 Julia 函数的 API，具体详情如下。

1.3.1 API 列表

函数名	简介
Julia	将 M 类型对象转换为 Julia 类型对象
fromJulia	将 Julia 类型对象转换为 M 类型对象
jeval	执行 Julia 代码，返回 Julia 对象
jcall	调用 Julia 函数返回 Julia 对象
jinterp	调用 Julia 函数返回 Julia 对象，支持结构体类型参数输入
jindex	获取 Julia 对象的索引，返回 Julia 对象
jisa	判断 Julia 对象数据类型

1.3.1.1 Julia

A. 语法

```
jv = Julia(o)
jv = Julia(o, jtype)
```

B. 说明

jv = Julia(o)，o 为 M 类型对象，将 o 转为 Julia 类型对象。

jv = Julia(o, jtype)，jtype 为字符串表示的 Julia 类型，将 o 转为 类型为 jtype 的 Julia 类型对象。

C. 示例

● 示例 1：创建未写明具体类型的 Julia 类型对象

```
% 未写明具体类型，数值默认转为 Array{Float64}
jv1 = Julia(1)
```

```
jv =
[1.0;]
```

```
% jisa 函数判断 jv 的类型
jisa(jv1,"Array{Float64}")
```

```
ans =
1x1 logical array
true
```

```
% cell 默认转为 Tuple，其中的数值元素转为 Array{Float64}
jv2 = Julia({1,2})
```

```
jv2 =
```

```

([1.0;:], [2.0;:])

% jisa 函数判断 jv 的类型
jisa(jv2, "Tuple{Array{Float64},Array{Float64}}")

ans =

1x1 logical array

true

% 创建 M 类型 struct 并转为 Julia 类型对象
jv = Julia(struct('name', 'a', 'value', 2))

jv =

(name = "a", value = [2.0;:])

% 判断 jv 是否为 NamedTuple 类型
jisa(jv, "NamedTuple")

ans =

1x1 logical array

true

```

● 示例 2：创建写明具体类型的 Julia 类型对象

```

% 创建 Int 类型的 Julia 标量
jv1 = Julia(1, "Int")

jv =

1

% 创建 Vector{Number} 类型的 Julia 向量
% Vector{Number} 实际上是 Vector{<:Number}
jv2 = Julia([1,2,3], "Vector{Number}")

jv2 =

[1.0, 2.0, 3.0]

% 将 M 元胞类型转为 Julia 元组类型
jv3 = Julia({1,2}, "Tuple{Int,Int}")

jv3 =

(1, 2)

```

1.3.1.2 fromJulia

A. 语法

```

o = fromJulia(jv)
o = fromJulia(jv, mtype)

```

B. 说明

`o = fromJulia(jv)`, `jv` 为 Julia 类型对象, 将 `jv` 转为 `M` 类型对象。

`o = fromJulia(jv, mtype)`, `mtype` 为字符串表示的 `M` 类型, 将 `jv` 转为类型为 `mtype` 的 `M` 类型对象。

C. 示例

● 示例 1：创建未写明具体类型的 M 类型对象

```
jv = Julia([1,2,3], "Array{Int64}");
% Array{Int64} 转为对应的 int64 类型
o = fromJulia(jv)
```

```
o =
1x3 int64 array
```

```
1 2 3
```

```
% 查看对象 o 的数据类型
class(o)
```

```
ans =
"int64"
```

● 示例 2：创建写明具体类型的 M 类型对象

```
jv = Julia([1,2,3], "Vector{Int}");
o = fromJulia(jv,"double");
```

```
o =
1 2 3
```

```
% 查看对象 o 的数据类型
class(o)
```

```
ans =
"double"
```

● 示例 3：将 Julia 的 Tuple 类型对象转为 M 类型对象

```
% 构造 Julia 的 Tuple 类型对象
jv = Julia({1,2}, "Tuple{Float64,Float64}");
% Tuple 转为 cell，Tuple 中 Float64 类型转为 double 类型
o = fromJulia(jv)
```

```
o =
1x2 cell array
{[1]} {[2]}
```

```
% 查看 o 的类型
class(o)
```

```
ans =
"cell"
```

```
% 查看 o 中元素的类型
class(o{1})
```

```
ans =
"double"
```

1.3.1.3 jeval

A. 语法

```
jeval(s)
```

B. 说明

jeval(s), s 为 Julia 代码字符串, 执行 Julia 代码, 返回 Julia 对象。

C. 示例

● 示例 1: 在 M 中执行 Julia 代码

```
jfunc1 = jeval('Float64')
```

```
jfunc1 =
```

```
Float64
```

```
jfunc2 = jeval('1 + 1')
```

```
jfunc2 =
```

```
2
```

1.3.1.4 jcall

A. 语法

```
jcall(expr, jv1, ..., jvn)
jcall(____, '-kwargs', kws)
jcall('-return', mtype, __)
jcall('-vector', __)
jcall('-noreturn', __)
```

B. 说明

jcall(expr, jv1, ..., jvn), expr 为 Julia 函数字符串, 调用 Julia 函数返回 Julia 对象。

jcall(____, '-kwargs', kws), expr 为 关键字参数结构体。

jcall('-return', mtype, __), mtype 为返回值类型字符串。

jcall('-vector', __), 向量化调用 Julia 函数。

jcall('-noreturn', __), 不返回值。

C. 示例

● 示例 1: Julia 函数的输入参数为单个参数

```
% 创建 类型为 Vector{Int} 的 Julia 对象
A = Julia([2,1,3],"Vector{Int}");
```

```
% 调用 Julia 函数需输入 Julia 对象
jv1 = jcall("sort",A)
```

```
jv =
```

```
[1, 2, 3]
```

```
class(jv1)
```

```
ans =
```

```
"julia_object"
```

```
% 将输出转为 M 对象
```

```
value1 = fromJulia(jv1,"double")
```

```
value =
```

```
1 2 3
```

```
class(value1)
```

```
ans =
```

```
"double"
```

● 示例 2: Julia 函数的输入参数包含关键字参数

```
B = Julia([1 2; 4 3],"Array{Int}");
```

```
% 关键字参数为结构体，字段名为 Julia 函数中关键字的 key 值
```

```
jkwargs.dims = Julia(2,"Int");
```

```
% '-kwards' 标识关键字参数，'-return' 标识输出参数类型
```

```
value2 = jcall('-return', 'double', 'sort', B, '-kwards', jkwargs)
```

```
value2 =
```

```
1 2
```

```
3 4
```

```
class(value2)
```

```
ans =
```

```
"double"
```

● 示例 3: 向量化调用 Julia 函数

```
C = Julia([pi 2*pi],"Vector{Float64}");
```

```
% '-vector' 标识向量化调用函数，'-return' 标识输出参数类型
```

```
value3 = jcall('-vector', '-return', 'double', 'cos', C)
```

```
value3 =
```

```
-1 1
```

● 示例 4: jeval 执行 Julia 代码

```
jfunc = jeval('(x, y) -> x - y');
```

```
x = Julia(1, "Int");
```

```
y = Julia(2, "Int");
```

```
% jfunc 代替 Julia 函数执行代码
value4 = jcall('-return', 'double', jfunc, x, y)
```

```
value4 =
```

```
-1
```

● 示例 5：调用 Julia 函数但不返回值

```
jcall('-noreturn', 'println', Julia("jcall noreturn", "String"));
```

```
jcall noreturn
```

1.3.1.5 jinterp

A. 语法

```
jinterp(expr, opts)
```

B. 说明

jinterp(expr, opts)，opts 是 M 结构体，其中的字段为 Julia 对象。

C. 示例

● 示例 1：Julia 函数的输入参数为多个参数

```
% 创建 opts 结构体
opts.X = Julia([3,2,1], "Vector{Int}");
jv = jinterp("sort(X)", opts)
```

```
jv =
```

```
[1, 2, 3]
```

```
% 将 Julia 类型对象转为 M 类型对象
value = fromJulia(jv, 'double')
```

```
value =
```

```
1 2 3
```

1.3.1.6 jindex

A. 语法

```
jvi = jindex(jv, {i1, ..., in})
jvi = jindex(jv1, {i1, ..., in}, jv2)
```

B. 说明

jvi = jindex(jv, {i1, ..., in})，获取 Julia 对象的索引，返回 Julia 对象。

`jvi = jindex(jv1, {i1, ..., in}, jv2)`, 设置 Julia 对象的索引。

C. 示例

● 示例 1: Julia 对象的索引操作

```
jv = Julia([1,2; 3,4],"Array{Int}");
```

```
% 获取索引为 2 的 Julia 对象
```

```
jv1 = jindex(jv, 2)
```

```
jv1 =
```

```
3
```

```
% 其类型为 Julia 类型
```

```
class(jv1)
```

```
ans =
```

```
"julia_object"
```

```
% 设置 Julia 对象的索引
```

```
jindex(jv,1 ,Julia(4,"Int"));
```

```
% 查看 jv 的值
```

```
jv
```

```
jv =
```

```
[4 2; 3 4]
```

1.3.1.7 jisa

A. 语法

```
jisa(jv, jtype)
```

B. 说明

`jisa(jv, jtype)`, 判断类型, 返回 logical, `jv` 为 Julia 类型对象, `jt` 为字符串或 `julia_object`。

C. 示例

● 示例 1: Julia 类型判断

```
jv = Julia(1, 'Int');
```

```
% 判断 jv 是否为 Number 类型的 Julia 对象
```

```
is_number = jisa(jv, 'Number')
```

```
is_number =
```

```
1x1 logical array
```

```
true
```

```
% 判断 jv 是否为 Real 类型的 Julia 对象
is_real = jisa(jv, jeval('Real'))
```

```
is_real =
1x1 logical array
true
```

1.3.2 M 类型与 Julia 类型对应关系

M 类型转为 Julia 类型之间的对应关系如下表所示：

表 1-3 M 类型转为 Julia 类型

M 类型	支持转换的 Julia 类型
double	Int/ComplexF64/Float64，及它们构成的数组
string	String, Vector{String}及其他 Array{String}
logical	Bool, Vector{Bool}及 Array{Bool}
uint8	UInt8, Vector{UInt8}及 Array{UInt8}
uint32	UInt32, Vector{UInt32}及 Array{UInt32}
uint64	UInt64, Vector{UInt64}及 Array{UInt64}
cell	Tuple
自定义映射的 M 类	自定义映射 M 类绑定的 Julia 类型，以及它的 Array 类型

例如：M 类型为 double，则可以将 其 转 为 Julia 的 Float64/Vector{Float64}/Array{Float64}类型。

若使用 Julia(o) 函数直接创建 Julia 对象时，遵循以下规则：

- 数值、数值数组类型默认转为 Array{Float64} 类型。
- string 类型默认转为 Array{String} 类型。
- struct 类型默认转为 NamedTuple 类型。
- cell 类型默认转为 Tuple 类型， cell 中的元素按照前述规则转换。

1.3.3 Julia 类型与 M 类型对应关系

Julia 类型转为 M 类型之间的对应关系如下表所示：

表 1-4 Julia 类型转为 M 类型

M 类型	支持转换的 Julia 类型
double	Int/ComplexF64/Float64，及它们构成的数组
string	String, Vector{String}及其他 Array{String}
logical	Bool, Vector{Bool}及 Array{Bool}

uint8	UInt8, Vector{UInt8} 及 Array{UInt8}
uint32	UInt32, Vector{UInt32} 及 Array{UInt32}
uint64	UInt64, Vector{UInt64} 及 Array{UInt64}
cell	Tuple
自定义映射的 M 类	自定义映射 M 类绑定的 Julia 类型，以及它的 Array 类型

2. 模型库开发规范

系统建模仿真环境中的模型库由一系列模块组成，模块是构建系统模型的主要元素。系统建模仿真环境内置模型库提供了机、电、液、控、热、磁等多学科基本模块，如果内置库未提供合适的模块，系统建模仿真环境允许用户开发新的模块并以模型库的方式集成到系统建模仿真环境中，从而扩展系统建模仿真环境功能。为保证各工程师开发的模型库质量、稳定性、可读性一致，需有对应规范来进行约束。本规范包括模型库开发规范和外部库封装规范。

2.1 模型库开发规范

2.1.1 模型库组织结构规范

清晰的模型库层次架构，能够让模型库具有很强的可重用性和重构性，能够全面直观的展示模型库信息，方便用户在此基础上进行快速的应用、完善和拓展。

架构设计中引入 Package 的概念帮助组织 Modelica 模型中的 models、connectors 等元件，将相关的模型有机地组合在一起。将一个领域中的元件及接口组织在一起的 Package 就叫做“库”。这些库往往是以一个整体的形式被引用，但是库中总是包含若干个子模型库，就模型库的逻辑结构内容，可参考如下：

表 2-1 参考模型库逻辑结构

名称		描述	说明
UsersGuide		用户指导	模型库介绍、版本信息、作者信息及其他信息。
Common (可分开)	Types	类型	存放模型库中所有自定义类型
	Interfaces	接口	模型接口，存放 connector 连接器模型
	Functions	函数	存放模型库中所有函数

	Icons	图标	存放模型库中所有模型图标
	Resources	外部函数	包含 Include、Library 子目录存放外部函数用以集成 Fortran、C 代码，以及 lib、dll 等函数库
AeroPowerSystem (视开发内容而定)		航空动力装置模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
ControlSystem (视开发内容而定)		控制系统模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
HydraulicSystem (视开发内容而定)		液压系统模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
...		...	...
Experiments		示例	系统模型示例集，主要包括各种仿真工况下的仿真模型
Tests		测试	模型测试用例，用于模型开发过程中的测试，在模型库交付时移除该内容

以航空发动机多物理领域模型库为例模型库结构如下图 2-1 所示：

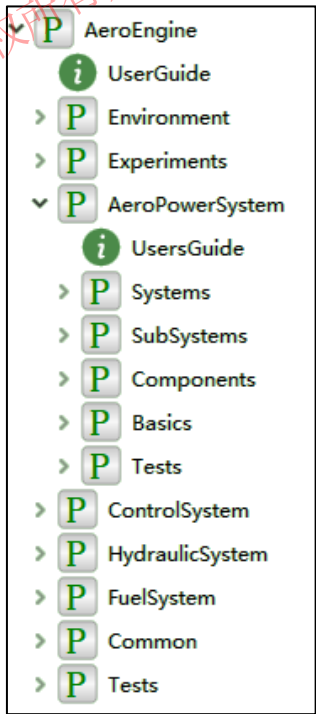


图 2-1 航空发动机多领域模型库

当模型依赖外部资源时，如图片、数据文件、C 代码等，需要保证模型对

资源的引用路径为相对路径，建议在模型库 Package 文件夹下新建 Resources 文件，并按类别存放对应的文件，如下为外部动态库的存放示例。

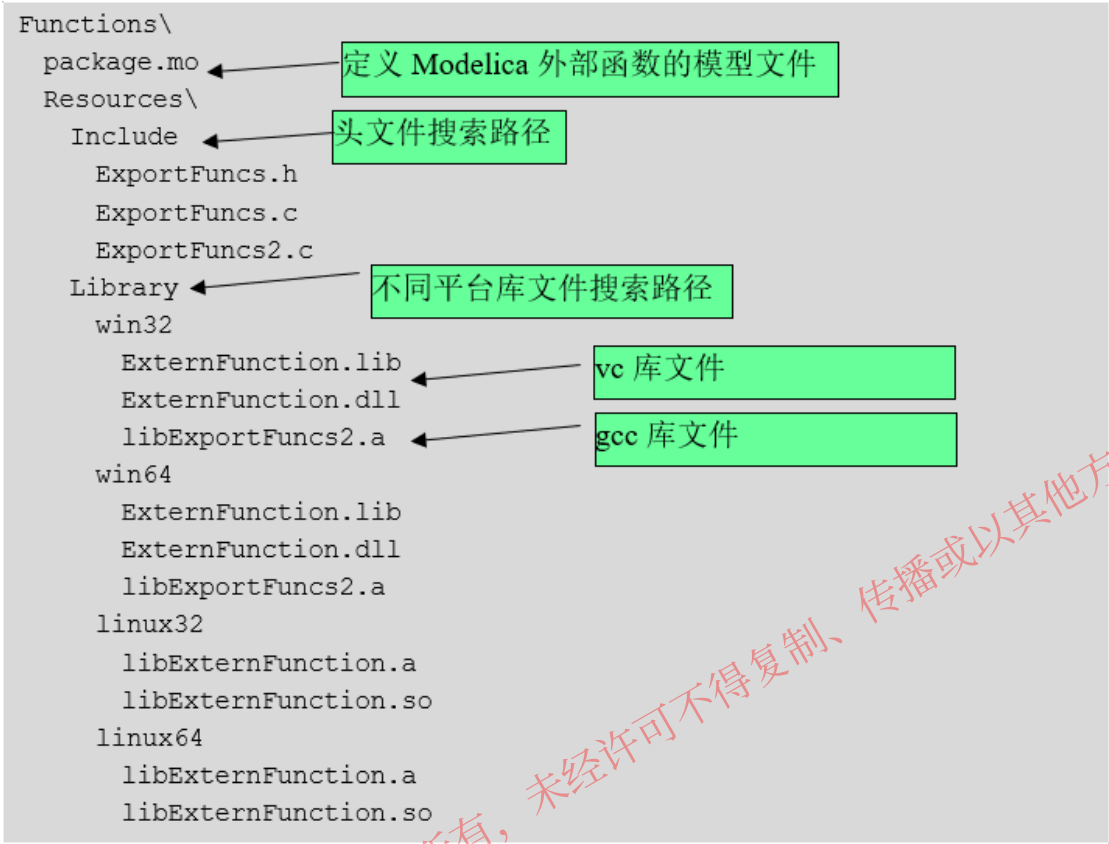


图 2-2 外部资源组织示例

2.1.2 模型编码规范

2.1.2.1 命名

命名需要在减少命名冲突的基础上增强模型代码的可读性。

减少命名冲突：建模过程中需要定义的参量、变量及系统自带的标识符和关键字之间应避免命名冲突。如：A 常用于表示面积、截面积等，因而在整个模型库中应该尽量减少字母 A 被用于其他含义的变量命名。

增强模型代码的可读性：对参数和变量的命名，除了可以方便地进行方程代码的书写，实现物理模型的行为外，还需要能够使读者快速理解该参数、变量的含义或者作用，更快地理解整个模型，因此其参数、变量的命名最好与其定义相对应，如长度的命名常常用单词 length 或者直接用大写字母 L 表示。

- 命名规则如下：
- (1). 类命名

- 对于一个或多个单词全拼组成的类名，每个单词首字母应大写，如 Example、ExamplePackeg、PartialModel 等；
- 对于存在大写简写元素的类，为避免引起歧义，每个独立元素之间用下划线连接，如 AD_Conversion（数模转换器）等；
- 对于类的实例化。首字母应小写，其它遵循以上原则，如 example、examplePackage 等；

(2). 参数或变量命名

- 变量的名字尽量能显示出变量的含义；
- 对于一个单词的参数或变量名，一般均采用小写字母，如 height、area 等；
- 对于多个单词组成的参数或变量名，每个单词之间用下划线隔开，angles_start、real_time 等；
- 对于只有一个字母的参数或变量名，其命名需根据实际情况确定，如：T（温度）、I(转动惯量)、t（时间）等；

(3). 连接器命名

- 对于标准库（Modelica3.2 标准库及以上版本）中已存在的连接器，可直接继承使用，无需再另行定义，如：电气 Pin；机械 Flange_a、Flange_b 等；
- 对于需新定义的连接器及变量命名，须遵循类命名、参数和变量命名规则；

(4). 常用航空术语命名规范

航空领域有着自己的专业术语与缩写，收集此类名词并合理运用到 Modelica 建模中，可以避免命名过于冗长，使 Modelica 模型库更为专业化，参见《航空专业英语缩写索引》、《英汉航空航天工程词典》及相关设计规范文档，具体根据项目研发定制统一的命名规范。

表 2-2 常用符号命名规范

缩写	解释	全称
APU	辅助动力装置	Auxiliary Power Unit
ATS	自动油门系统	Autothrottle System
EEC	发动机电子控制器	Electronic Engine Control
EGT	排气温度	Exhaust Gas Temperature
.....		

2.1.2.2 模型行为和方程

对于一个物理系统而言，各组成部分内部及各组成部分之间的物理关系本身并没有因果性，因此，模型中应当是对模型行为的自然描述而无需考虑计算顺序。这种基于方程的建模方式称为陈述式建模，又称为非因果建模，与之相对的是过程式建模或因果性建模。

Modelica 是基于方程的建模语言，在上述的类型定义中可以看到，中间有明确的行为区域用于以方程来描述模型行为，作为对陈述式建模的补充，Modelica 同时支持过程式建模方法。

陈述式建模的特点之一是通过方程而非赋值来描述模型行为。Modelica 支持陈述式/过程式混合建模，分别以 `equation` 和 `algorithm` 来分别描述方程和赋值，区别在于，`equation` 定义的方程组间的求解顺序将由软件推断确定，而 `algorithm` 定义的则按照定义顺序进行求解。例如：

陈述式建模	过程式建模
<pre>model BasedOnEquation Real x; Real y; Real z; equation x + y + z = 1; x + 2 * z = y; x - y = time; end BasedOnEquation;</pre>	<pre>model BaseOnAlgorithm Real x; Real y; Real z; algorithm z := -time / 2; y := (1 - z - time) / 2; x := 1 - y - z; end BaseOnAlgorithm;</pre>

陈述式建模的最大优点是：用户建模时只需专注于物理问题的陈述，而无需考虑物理问题的错综复杂的求解过程怎样实现，因而建模更加简单，所建模型更加健壮。

对一个物理系统而言，其各组成部分内部及各组成部分之间的物理关系本身就是非因果的，这与 Modelica 语言倡导的陈述式建模理念是完全吻合的。

2.1.2.3 代码结构

为了实现模型库编码风格的统一，模型库创建过程中应遵循如下的 Modelica 模型编码顺序：

- 第一部分：继承类语句，如 `import`、`extend`、`outer` 等；
- 第二部分：模型参数，`parameter`；
- 第三部分：模型变量；
- 第四部分：模型接口，`FluidPort_a/flange_a`；
- 第五部分：初始方程，`initial equation`；

- 第六部分：方程和算法，equation、algorithm。

具体代码结构示例如下形式：

`model FixedDoubleActingCylinder` "缸体固定式双作用液压缸"

//第一部分：继承类语句如import、extend、outer等；

```
extends Utilities.Icons.FixedDoubleActingCylinder2;
extends Components.BasicModels.TwoPortsCylinder_piston;
```

//第二部分：模型参数，parameter；

```
parameter SI.Diameter D = 0.05 "活塞直径"
    annotation (Dialog(tab = "结构参数",group = "基本参数"));
parameter SI.Diameter d = 0.024 "活塞杆直径"
    annotation (Dialog(tab = "结构参数",group = "基本参数"));
.....
```

//第三部分：模型变量；

```
SI.Length s(start = initialPosition) "活塞位移";
SI.Velocity v "活塞运动速度";
SI.Acceleration a "活塞运动加速度";
.....
```

//第四部分：模型的接口；

```
Modelica.Mechanics.Translational.Interfaces.Flange_a flange_a 固体接口
    annotation (...);
.....
```

//第五部分：初始方程，initial equation；

```
initial equation
    v = 0 "活塞初始速度";
```

//第六部分：方程和算法，equation、algorithm；

```
equation
    //速度、加速度方程
    v = der(s);
    a = der(v);
    //左右腔等效容积
    effVolume_L = activeArea_L * s + deadVolume;
    effVolume_R = activeArea_R * (L - s) + deadVolume;
    .....
```

`end FixedDoubleActingCylinder;`

2.1.2.4 注释

建立的所有类别（package, model, function）都需要相应的注释，一般类注释普遍采用下面的形式，一般用“`"""`”进行注释，如：

```
package AeroEngine_HyMo "航空发动机液压元部件模型库"
    model Pipe "管道"
        .....
    end Pipe;
    function ReynoldsNumber "雷诺数函数"
        .....
    end ReynoldsNumber;
end AeroEngine_HyMo;
```

(1). 参变量注释

定义的每个参量，变量都要注释该参变量的含义，增强代码的可读性，注释语尽量简明扼要，一般用“`"""`”进行注释，如：

```
parameter SI.Length L = 1 "管道长度";
parameter SI.Diameter d = 0.02 "管道内径";
SI.Area A = d ^ 2 * Constants.pi / 4 "管截面积";
SI.Volume V = L * A "管道容积";
```

(2). 方程注释

在模型代码的方程区域，某一个方程的含义或某一部分方程的含义都要注释清楚，一般用“`//`”进行注释，如：

```
Equation
//流量方程
m_flow = sign(dp) * A * Cq * sqrt(2 * rho * noEvent(abs(dp)));
```

2.1.2.5 参数及参数框设计

A. 参数定义

定义模型参数包括参数数据类型、名称、默认值、量纲、必要的描述信息等，参数示例如下：

表 2-3 参数定义

参数类型	参数名称/默认值/量纲	参数说明
Modelica.SIunits.Length	Stroke = 0.03	换向阀行程
Boolean	Init=false	阀芯初始位置是否由输入决定

Real	Cq=0.7	流量系数
...	...	...

B. 参数框定义

参数框中的参数显示都是按照“先定义先显示”的原则，包括 Tab/Group 的显示，所以参数框的显示最好按照参数的“重要程度”进行有区别显示，进行参数分类显示时一般遵循这样几点原则：

- (1). 参数少，分 Group；
- (2). 参数多，分 Tab；
- (3). 重要共性参数（介质）单独分开，Tab/Group 统一；
- (4). 参数的一般分类：
 - a. 工质参数
 - b. 结构参数
 - c. 初始条件（变量初始化及初始参数）
 - d.

参数框设计代码：

```
parameter SI.Diameter D = 0.05 "活塞直径"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Diameter d = 0.024 "活塞杆直径"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Mass m = 0.5 "活塞质量"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Length L = 2 "活塞行程"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.TranslationalDampingConstant c = 2000 "活塞运动阻尼"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Volume deadVolume = 1e-6 "死区容积"
  annotation (Dialog(tab = "限位参数"));
parameter SI.TranslationalSpringConstant c_c = 1e8 "接触刚度"
  annotation (Dialog(tab = "限位参数"));
parameter SI.TranslationalDampingConstant d_c = 1e6 "接触阻尼"
  annotation (Dialog(tab = "限位参数"));
parameter SI.Distance initialPosition = 0 "活塞初始位置"
  annotation (Dialog(tab = "初始参数"));
```

参数框显示结果：

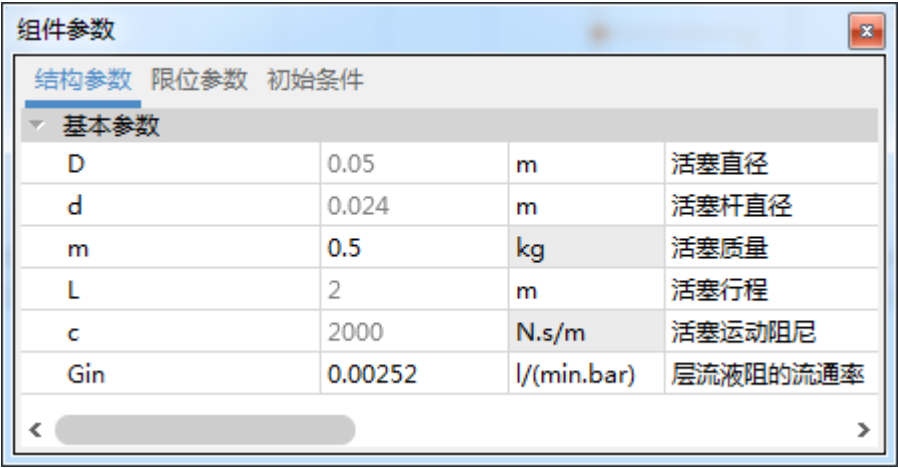


图 2-3 参数框设计参考

2.1.2.6 接口定义

为避免系统模型集成阶段由接口带来的问题，在系统分析设计初期就需要将接口定义清晰，以及各系统间的连接方式。

接口说明内容包括：模型输入/输出变量的数据类型、维数、名称、量纲、必要的描述信息，接口说明示例如下：

表 2-4 接口定义

接口	数据类型	名称	量纲	描述	其他说明
输入	Real[1,1]	H	m	飞行高度	高度表接口
	Real[1,1]	Ma	1	飞行马赫数	空速表接口
	Real[1,1]	TLADeg	deg	油门杆角度	
	...	...	...	...	...
输出	Real[1,1]	qm_f	kg/s	燃油流量	
	...	...	...	...	...

针对特定行业接口主要包含机械接口、电接口、热接口、信息接口和流体接口。

A. 机械特性接口

表 2-5 模型输入接口

	flow 变量	flow 变量	势变量	势变量
变量类型	Force	Torque	Position	Frames.Orientation
变量名称	f[3]	t[3]	r_0[3]	R

描述	剪切力	剪切力矩	从世界坐标系原点到连接点的位置向量 (相对世界坐标系)	将世界坐标系转到连接点的方向向量
功能	机械多体接口, 描述了剪切力、剪切力矩和两个方向向量			

表 2-6 模型输出接口

	flow 变量	flow 变量	势变量	势变量
变量类型	Force	Torque	Position	Frames.Orientation
变量名称	f[3]	t[3]	r_0[3]	R
描述	剪切力	剪切力矩	从世界坐标系原点到连接点的位置向量 (相对世界坐标系)	将世界坐标系转到连接点的方向向量
功能	机械多体接口, 描述了剪切力、剪切力矩和两个方向向量			

B. 电特性接口

表 2-7 模型输入接口

	flow 变量	势变量
变量类型	Current	Voltage
变量名称	i	v
描述	电流	电压
功能	用于基本模型中电流和电压的传递	

表 2-8 模型输出接口

	flow 变量	势变量
变量类型	Current	Voltage
变量名称	i	v
描述	电流	电压
功能	用于基本模型中电流和电压的传递	

C. 热特性接口

表 2-9 模型输入接口

	flow 变量	势变量
变量类型	HeatFlowRate	Temperature
变量名称	Q_flow	T
描述	接口热流量	接口温度
功能	用于基本模型中热流量和温度的传递	

表 2-10 模型输出接口

	flow 变量	势变量
变量类型	HeatFlowRate	Temperature
变量名称	Q_flow	T
描述	接口热流量	接口温度

功能	用于基本模型中热流量和温度的传递
----	------------------

D. 信息特性接口

表 2-11 模型输入接口

	变量类型	变量名称	描述	功能
RealInput	Real	input	实型信号输入接口	实型数据传递
RealOutput	Real	output	型信号输出接口	
IntegerInput	Integer	input	整型信号输入接口	整型数据传递
IntegerOutput	Integer	output	整型信号输出接口	
BooleanInput	Boolean	input	布尔型信号输入接口	布尔型数据传递
BooleanOutput	Boolean	output	布尔型信号输出接口	

表 2-12 模型输出接口

	变量类型	变量名称	描述	功能
RealInput	Real	input	实型信号输入接口	实型数据传递
RealOutput	Real	output	型信号输出接口	
IntegerInput	Integer	input	整型信号输入接口	整型数据传递
IntegerOutput	Integer	output	整型信号输出接口	
BooleanInput	Boolean	input	布尔型信号输入接口	布尔型数据传递
BooleanOutput	Boolean	output	布尔型信号输出接口	

E. 流体特性接口

表 2-13 模型输入接口

	flow 变量	势变量	Stream 变量	Stream 变量
变量类型	VolumeFlowRate	AbsolutePressure	SpecificEnthalpy	MassFraction
变量名称	w	p	h	Xi
描述	接口质量流	接口压力	接口比焓	接口组分质量分数
功能	一维流体接口，用于基本模型中流动工质的质量流、压力、比焓和组分的传递			

表 2-14 模型输出接口

	flow 变量	势变量	Stream 变量	Stream 变量
变量类型	VolumeFlowRate	AbsolutePressure	SpecificEnthalpy	MassFraction
变量名称	w	p	h	Xi
描述	接口质量流	接口压力	接口比焓	接口组分质量分数
功能	一维流体接口，用于基本模型中流动工质的质量流、压力、比焓和组分的传递			

2.1.3 模型图形布局规范

基于组件连接构建系统模型时，应对系统模型进行合理布局，兼顾系统模型的可读性与美观性。注意以下几点：

- (1). 能显示出系统的层次结构，可用画图工具将重要区域标注，并附上简单的文字说明；
 - (2). 合理利用空间，各组件连接紧凑，互不干涉；
 - (3). 尽量使系统模型结构整体看起来赏心悦目，并显示出其专业性。
- 可以参考如下模型的布局：

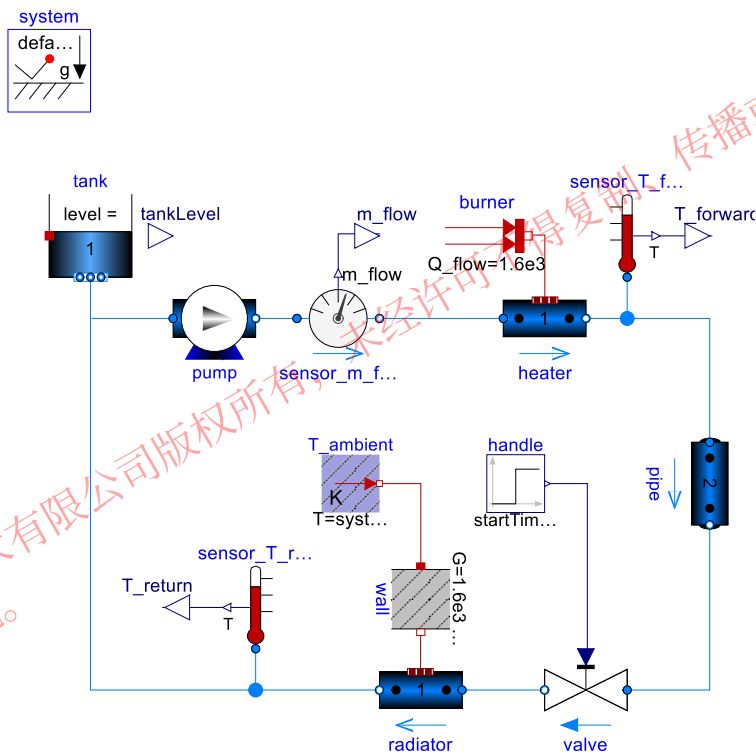


图 2-4 简单加热系统模型布局

2.1.4 模型用户指南编写规范

新建的模型库信息一般都是通过建立 UsersGuide 库进行相应地说明，一般模型库的 UsersGuide 主要分为这样几部分：

- (1). Overview（综述）——模型库特点，构架等；
- (2). UsersGuide（用户指导）——使用说明，开发说明等；
- (3). ReleaseNotes（版本日志）——版本修订日志；

- (4). Help（帮助）——关联用户说明手册或者 FAQ；
- (5). References（参考资料）——模型库建模主要参考资料；
- (6). Contant（联系方式）——开发者的相关信息。

对于任何一个组件模型及系统模型，应给出详尽的说明文档，清楚明了的说明该模型的功能、用法及注意事项。从使用者角度出发对模型简介、模型原理、模型接口、模型参数等方面进行描述，可参见附录 A。

- 简介，对模型的功能及用法进行简要描述；
- 原理，阐述模型的结构与原理，以及模型开发所应用到的理论基础、理论公式等信息；
- 接口，包括模型的输入、输出接口的类型、描述信息；
- 参数，包括模型参数的类型、量纲、描述信息。

2.1.5 模型工程化实践规范

2.1.5.1 改善模型健壮性

模型编码时需要注意模型的健壮性，不仅要每个元件进行充分的测试，也需要防止库被误用。

- 输出限幅，最大值、最小值限制等，可使用断言语句或加入限幅组件模型等，如：`assert(i*v<1e6,"Maxinum power exceeded")`；
- 使用 `final` 关键字和限制属性；
- 好的文档，给使用者意见和提醒，减少误用。

2.1.5.2 提高仿真效率

(1). 尽量使用 equation

如果没有特殊的目的，请尽量使用 `equation`。原因是在 `algorithm` 块中，变量可能被赋值多次，使得工具无法对其进行符号分析，提高仿真效率。例如，无法得到求解雅阁比矩阵的函数。

(2). 避免不必要的事件

事件会中断积分算法，从而影响仿真速度，因此避免不必要的事件可以较大地提供仿真速度。如：

```
der(x)= noEvent(if y<0 then 0 else y^2);
```

(3). 合适的积分算法

积分算法有自己的适用范围，因此需要模型的特点，采用合适的算法，例如模型是刚性的，则需要采用求解刚性问题的积分算法。

(4). 合适的误差

误差越小仿真时间越长，结果越可靠，而误差越大仿真速度越快，但结果越不可靠。可以使用不同的误差仿真，比较它们的结果，而确定合适的误差。

(5). 为函数提供雅阁比函数

对于隐式方程组，分析器使用符号推导，得到求解雅阁比矩阵的函数，避免数据求解。由于分析器不能推导用户自定义函数，因此如果用户为自定义函数提供导数函数，可以提高仿真效率。如：

```
model JacobianExample
  Real x,y;
  function f
    input Real x;
    output Real y;
    annotation (derivative=f_Jac);
  algorithm
    y := sin(x)+cos(x)+x;
  end f;
  function f_Jac
    input Real x;
    input Real dx;
    output Real dy;
  algorithm
    dy := dx * (cos(x)-sin(x)+1);
  end f_Jac;
equation
  der(y) = 2.0;
  y = f(x);
end JacobianExample;
```

(6). 变量消除

分析器会进行符号处理，消除别名变量。因此用户手动消除别名变量对仿真速度没有影响，而且容易使代码变得不直观。除非有证据表明消除变量对仿真速度有影响，否则不建议手动消除别名变量。如：

```
model Resistor
  import Modelica.SIunits;
  import Modelica.Electrical;
  SIunits.Voltage v "Voltage from pin p to n";
  SIunits.Current i "Current entering at pin p";
  Electrical.Analog.Interfaces.Pin p "Positive";
  Electrical.Analog.Interfaces.Pin n "Negative";
  parameter SIunits.Resistance R = 300 "Resistance";

equation
  v = p.v - n.v;
  0 = p.i + n.i;
  i = p.i;
  i * R = v;
end Resistor;
```

不建议消除

消除后的方程

$$R * p.i = p.v - n.v;$$
$$p.i + n.i = 0;$$

2.1.6 模型测试规范

模型测试规范包括模型测试验证流程和结果评价方法。模型测试验证流程用于规范模型测试过程，结果评价方法用于规范模型测试评价结果。

2.1.6.1 测试验证流程

模型验证的总体方法是模型仿真结果文件与可靠数据进行比较判断。其流程如图 2-5 所示：分别用模型库及系统的物理试验数据或（和）同类软件的仿真结果数据来和仿真结果数据作对比、分析，从而完成模型验证。

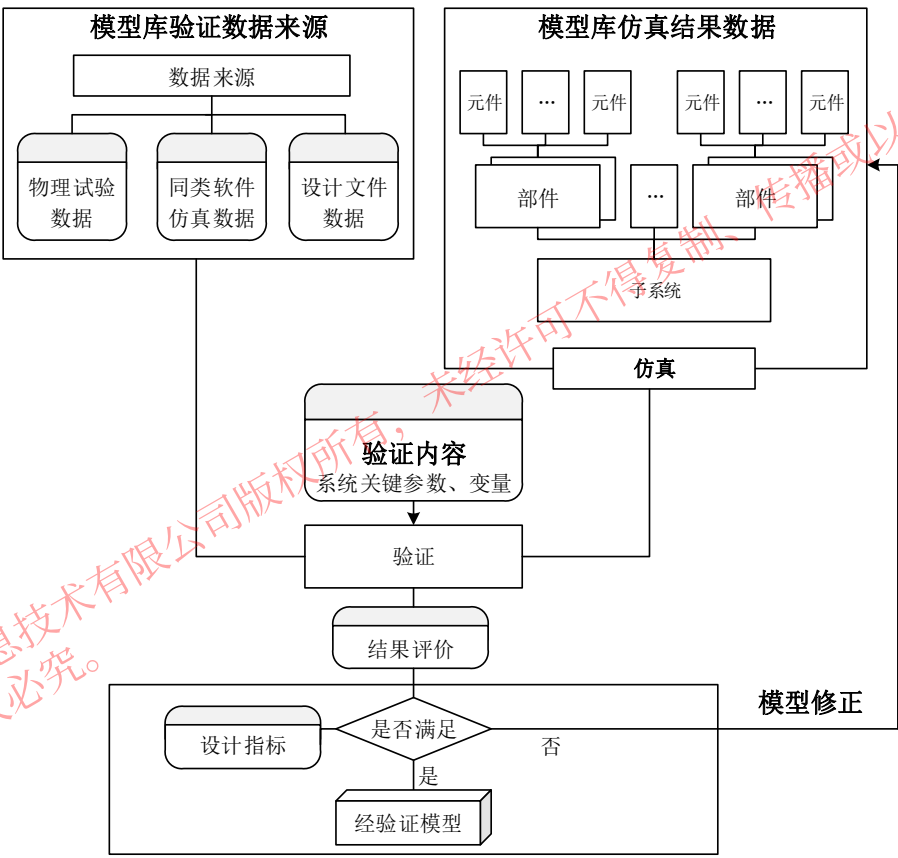


图 2-5 模型验证流程

对于以上模型验证流程，其数据来源包括三种，第一种为设计文件数据，第二种为同类软件仿真结果数据，第三种为物理试验数据。模型验证的层次分二层，第一层为模型库组件模型的验证，第二层为系统模型的验证。

2.1.6.2 结果评价

组件模型的测试需得到指定的关键数据结果，并且与对应物理试验结果进

行比较。具体结果评价方法可参考静态误差与动态误差评价方式。

A. 稳态误差

1) 评价方式

对于稳态误差的评价方法，下面提供几种误差衡量指标：

- 平均偏差：根据每个采样时刻的仿真值与实际值之间差值的绝对值与仿真值之间的比值来计算误差百分比，最后通过加权的方式来获取整批数据的误差：

$$F = \frac{1}{n} \cdot \sum_{i=1}^n \left(\frac{|X_{\text{仿真}}(i) - X_{\text{试验}}(i)|}{|X_{\text{仿真}}(i)|} \times 100\% \right)$$

- 最大误差：即为所有采样时刻的仿真值与实际值的最大值。

$$F = \max_{i=1, \dots, n} (|X_{\text{Measurement}}(i) - X_{\text{Simulation}}(i)|)$$

2) 应用场景示例

静态误差的使用适用于稳态过程，即系统的输出数据达到一个固定的稳态值时进行评价。适用于大多数模型的仿真结果评价。

B. 动态误差

1) 评价方式

对于动态误差的评价方法，即当输入数据为连续变化时，系统的输出数据没有一个固定的稳态值时，无法计算系统输出的稳态误差，此时，可考虑以累计误差为评价标准进行衡量。

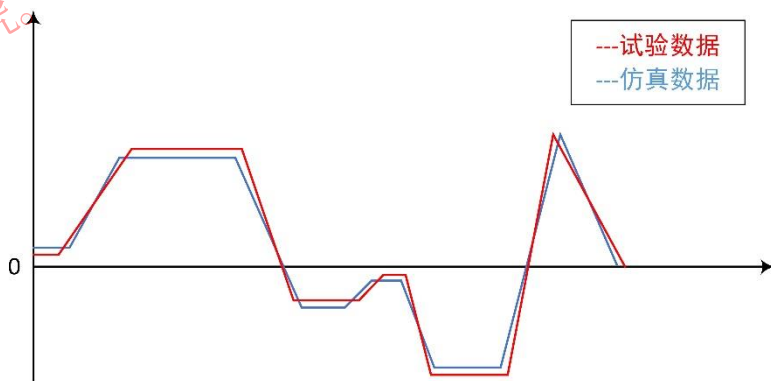


图 2-6 动态过程输入示意

- 累计误差：取试验、仿真曲线绝对值，通过积分的方式计算绝对值曲线面积与各采样点绝对误差曲线面积来反映误差大小，相当于将每个时刻的误差进行累计。累积误差计算方式如下：

$$F = \frac{S_{\text{误差}}}{S_{\text{试验}}} \times 100\% = \frac{\int |X_{\text{仿真}}(i) - X_{\text{试验}}(i)| \cdot \Delta t}{\int |X_{\text{试验}}(i)| \cdot \Delta t} \times 100\%$$

其中 $S_{\text{误差}}$ 为绝对误差曲线与时间轴所围成的面积， $S_{\text{试验}}$ 为试验曲线绝对值与时间轴所围成的面积， Δt 为采样时间， i 为对应每个采样的序号。

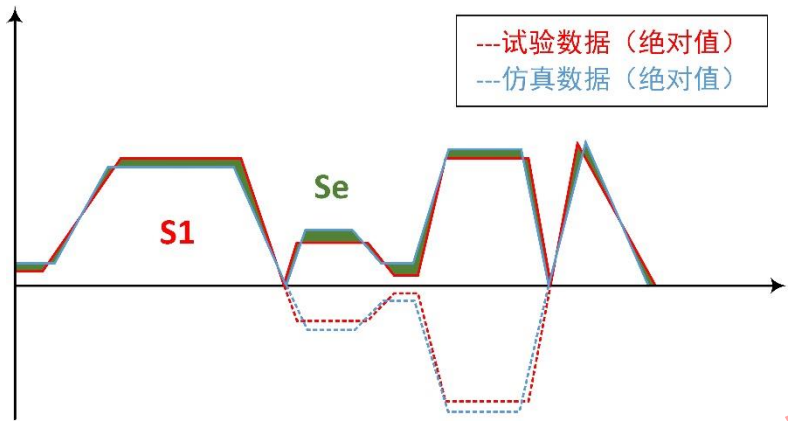


图 2-7 累积误差计算示意图

2) 应用场景示例

如在下列连续变化输入的动态仿真过程中，选择以累计误差评价动态误差的方式，比较合适。

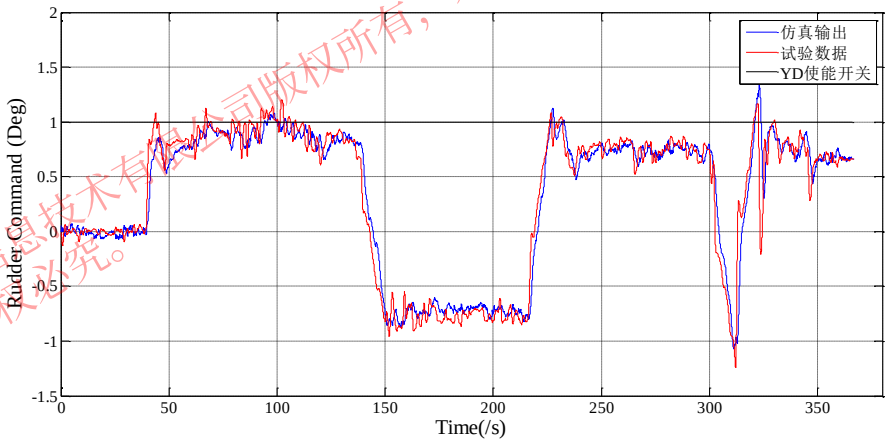


图 2-8 基于动态误差的结果评价应用场景

其误差计算结果为：

仿真绝对值曲线面积：239.7146

试验绝对值曲线面积：247.0526

误差计算：3.06%

2.1.7 模型库版本管理规范

在模型库构建以及后期维护过程中，存在大量的模型库版本迭代，应对模型库的版本进行管理，确保版本的准确性和可追溯性。

A. 版本命名

模型库版本命名主要由 5 部分组成，第一部分为模型库名称，第二部分为主版本号，第三部分为子版本号，第四部分为日期版本号和希腊字母版本号，第五部分为项目缩写名称。希腊字母版本号共有 4 种，分别为 Alpha、Beta、RC、Release。例如：HyMo_1_1_20180810_Release_BJHFYYJM。

- 模型库名称：指用户所构建的模型库名称，通常是系统物理属性等的英文全称或英文简写；
- 主版本号：当模型库由较大变动，如增加多个模块或这整体架构发生变化，此时可根据需要进行修改；
- 子版本号：当模型功能有一定增加或变化，如提高模型粒度等，可根据需要进行修改；
- 日期版本号：用于记录修改时间，每天对模型的修改都需要更改日期版本号；
- 希腊字母版本号
 - Alpha 版：主要指模型以实现功能为主，该版本模型存在较多问题，需进行修改；
 - Beta 版：较 Alpha 版修改了严重问题，但仍存在一定缺陷，需要经过多次测试来进一步消除；
 - RC 版：该版本相对成熟，已修正了绝大部分错误，与最终版本接近；
 - Release 版：经过多轮修改后的最终版本。

B. 版本升级

主要针对主版本号和子版本号，其基数均为 0 或 1，在每一次模型版本发生变更时，均在当前版本号上加 1，并以文本或其它形式标注每个版本号对应的变更内容。

2.2 外部语言封装规范

2.2.1 C/C++语言封装规范

用户可能采用 C/C++/Fortran/Python 等语言开发的模型，需要集成到本仿真平台中。Modelica 提供了外部函数机制，支持将外部语言开发的模块集成到 Modelica 模型库。

Modelica 中调用外部函数通过 Modelica function 进行，这种 Modelica 函数没有算法区域，取而代之的是外部函数接口声明语句“external”，以表示调用的是外部函数。

2.2.1.1 外部函数

模型中除了可以调用使用 Modelica 语言编写的函数外，还可以调用其它语言编写的函数，这些其它语言编写的函数称为外部函数。Modelica 中调用外部函数通过 Modelica 函数进行，这种 Modelica 函数没有算法（algorithm）区域，取而代之的是外部函数接口声明语句“external”，用以表示调用的是外部函数。

```
package myfuncs
function sin
  input Real x;
  output Real y;
  external "C";
end sin;
end myfuncs;
```

函数 myfuncs.sin 调用 C 语言函数 sin，调用形式为“y=sin(x)”。在 external 中可选地指定外部函数的语言、外部函数调用声明等。

注解用来指定数组布局（arrayLayout）是按行主存储（rowMajor）还是按列主存储（columnMajor）、外部函数所需的头文件（Include）、外部函数所需的库文件（Library）、外部函数所需头文件所在的位置（IncludeDirectory）、外部函数所需库文件所在的位置（LibraryDirectory）。

```
package myfuncs
function exp "e 的指数"
  input Real u;
  output Real y;
  external "C" y = exp(u) annotation (Include = "#include<math.h>");
end exp;
end myfuncs;
```

如果没有 IncludeDirectory 注解，外部函数所需头文件默认位置是包含外部函数的包(package)文件(“.mo”)所在文件夹中的“Resources/Include”子文件夹。同样，若没有 LibraryDirectory 注解，外部函数所需库文件默认位置是包含外部函

数的包(package)文件(“.mo”)所在文件夹中的“Resources/Library”子文件夹，不同平台的目标文件还可以存放在相应的平台文件夹中。

支持以下标准平台：

- win32 [32 位 Microsoft Windows]
- win64 [64 位 Microsoft Windows]
- linux32 [Intel 32 位 Linux]
- linux64 [Intel 64 位 Linux]

2.2.1.2 Modelica 模式 URI

Modelica 模式 URI 用于表示 Modelica 资源所在的位置，其形式如下：

```
modelica://[LibraryName]/[Path]
```

URI 由三部分组成：“modelica://”是 modelic 模式 URI 的专用标识，跟在其后两部分为 LibraryName 跟 Path：LibraryName 表示 Modelica 包的全名，path 部分用于表示资源相对于包的位置，是相对路径。如果 LibraryName 中有 Q-IDENT，则所有 URI 保留的字符都遵循 URI 表示的规则，即百分号编码。示例：

```
modelica://Modelica.Mechanics/Src/Include
```

LibraryName: Modelica.Mechanics, Path: Src/Include

```
modelica://myRec.%27%2b%27/Lib
```

LibraryName: myRec.'+', Path: Lib

2.2.1.3 Include 和 IncludeDirectory 注解

Include 注解中的内容直接嵌入至模型 C 代码中，因此，其中可以指定包含文件或者直接写一段 C 代码。示例：

指定包含文件

```
function exp "Exponential, base e"
  input Real u;
  output Real y;
  external "C" y = exp(u) annotation(Include="#include \"math.h\"");
end exp;
```

直接嵌入 C 代码：

```
function IncTest1
  input Real x;
  output Real y;
  external "C" y=myDouble(x) annotation(Include="double myDouble(double x)
                                     {return 2*x;}");
end IncTest1;
```

IncludeDirectory 注解指定包含文件所在的位置，以 URI 的 modelic 模式表

示。

2.2.1.4 Library 和 LibraryDirectory 注解

Modelica 中外部函数声明时用 Library 注解指定链接库名（注意：不带扩展名）。下面示例中指定的连接库名是 Fac，那么完整的连接库名称是 Fac.lib（vc 编译生成）或 libFac.a（gcc 编译生成）。

```
function Fac "factorial"
  input Integer n;
  output Real res;
  external "C" annotation (Library = "Fac",
    LibraryDirectory="modelica://ExternFuncUseDll.ExternFunc");
end Fac;
```

LibraryDirectory 注解指定库文件和 dll（或 so）文件所在的位置。指定位置中可以使用不同的平台文件夹存放各平台的库文件和 dll（或 so）文件，：

- win32 [32 位 Microsoft Windows]
- win64 [64 位 Microsoft Windows]
- linux32 [Intel 32 位 Linux]
- linux64 [Intel 64 位 Linux]

2.2.2 Julia 函数库集成规范

本节说明如何将科学计算环境中开发的 Julia 函数库封装为 Modelica 函数库中的模块，以支持用户在系统仿真环境中重用 Julia 函数库，并以图形化的方式集成科学计算函数或模型。

2.2.2.1 Syslab Function 开发规范

Syslab Function 是一种将 Julia 函数封装为 Modelica 模型的机制。Syslab Function 通过 Modelica 外部函数实现。本节说明 Syslab Function 的功能机制和操作步骤。

Syslab Function 包含两个模型：

- SyslabGlobalConfig：用于全局声明，包括导入包及全局变量声明等。
- SyslabFunction：用于嵌入 Julia 函数，并将 Syslab Function 模块的输入和输出数据指定为参数和返回值。

2.2.2.1.1 SyslabGlobalConfig

用于为系统中的 Julia 函数提供全局声明，例如导入包或声明全局变量。当

创建了 SyslabGlobalConfig 组件之后，点击右键菜单中的“Syslab 初始化配置...”可以在 Syslab 中打开编辑器，编写全局声明的 Julia 脚本。

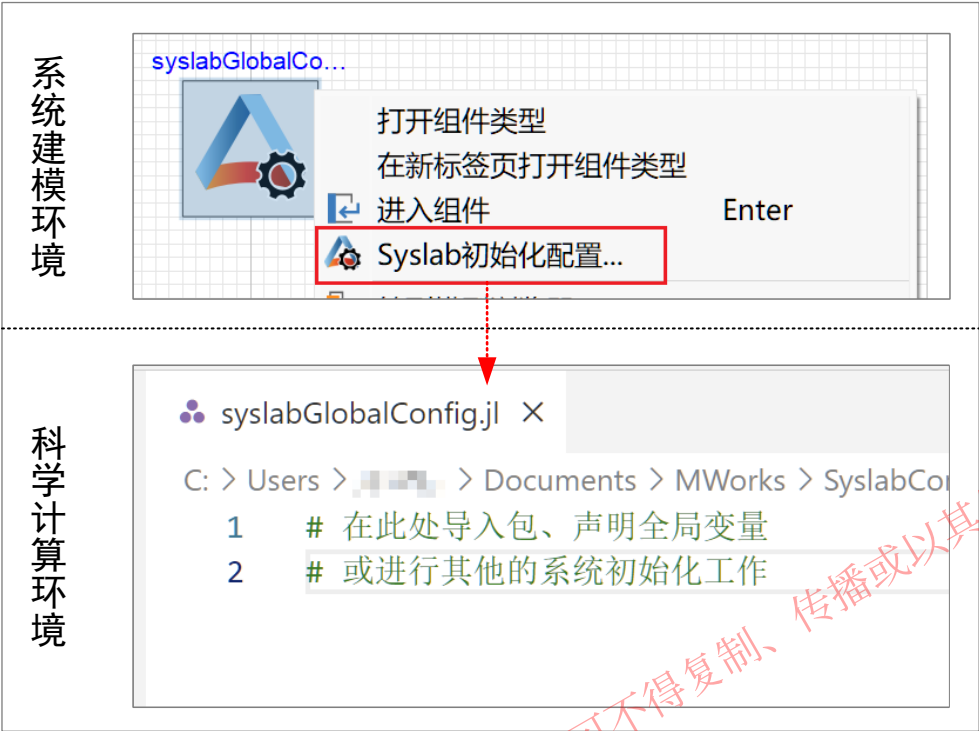


图 2-9 SyslabGlobalConfig

2.2.2.1.2 SyslabFunction

SyslabFunction 用于嵌入 Julia 函数，并将 Syslab Function 模块的输入和输出数据指定为参数和返回值。系统仿真每推进一步都会调用该 Julia 函数。

对于 SyslabFunction 组件而言，点击右键菜单中的“编辑 Syslab 脚本函数...”项，可以在 Syslab 中打开编辑器，编写 Julia 脚本，例如：

```
1 function func1(t)
2     x,y = get_xy(t)
3     return x,y
4 end
5
6 function get_xy(t)
7     a = [t, 2t]
8     b = [t 2t 3t; 4t 5t 6t]
9     return a,b
10 end
11
```

图 2-10 SyslabFunction

上图中的 Julia 脚本即完成了一个 Syslab Function 定义，组件将生成一个名

为 `in_t` 输入端口和两个分别名为 `out_x`、`out_y` 的输出端口。

SyslabFunction 组件认为脚本中的第一个函数为本组件的主函数，其他函数均为服务于主函数的辅助函数。根据主函数的内容，组件从函数声明中的输入参数中，获取组件的输入端口的数量及名称；从返回值语句中获取组件的输出端口的数量及名称。**Syslab** 的主函数的定义规则如下：

- 模型的输入端口数量由 **Julia** 函数的声明决定，类型和维度在端口由用户在设置对话框中指定；
- 模型的输出端口数量由 **Julia** 函数的返回值决定，类型和维度在端口由用户在设置对话框中指定；
- 端口类型支持实型、整型、布尔型，维度字段如果空缺，则认为端口为标量端口；
- 主函数必须使用 **function** 定义；
- 主函数的输入不要指定类型，不要指定具名参数；
- 主函数的输出必须使用 **return** 指定，且必须为函数体中已经出现的变量符号；
- 在同一个系统模型中，函数名必须是唯一的；

2.2.2.2 Syslab Object 开发规范

Syslab Object 是一种 **Julia** 类，具有用于算法建模的特定方法和属性。**System object** 适用于为动态系统建模和处理流式数据。用户可以使用模型库产品中包含的预定义 **Syslab Object**，也可以定义自己的 **Syslab Object**。

Syslab Object Block 是系统建模环境将 **Syslab Object** 封装成的 **Mo** 模块，提供了一套结构化的模板流程，可在执行期间自动完成许多管理操作，包括初始化系统、验证数据、单步计算和终止等操作。

Syslab Object 的组件关系如下图所示。

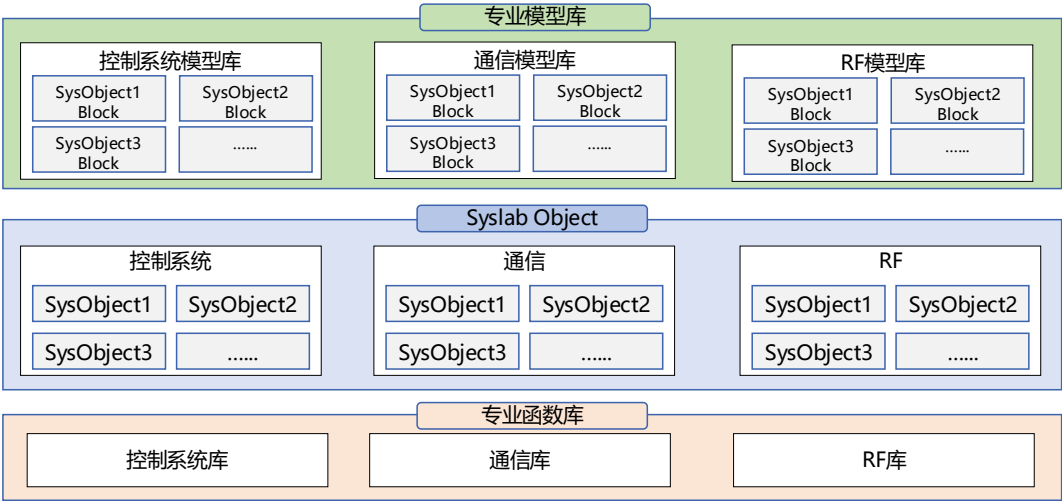


图 2-11 Syslab Object 组件关系图

2.2.2.2.1 Syslab Object 的开发规范

Syslab Object 是一个 Julia 类，包含类名、类属性、类方法等基本要素。Syslab Object 的开发需要遵循具体的开发规范，其模板文件如下所示。

```
using ObjectOriented
@oodef mutable struct Untitled
    # Description (用于生成 Modelica 组件描述)
    # Template for syslab object block

    # Parameter (用于生成 Modelica 组件参数，包括名称、类型、描述)
    # 格式：参数名::参数类型 = 参数值 # 参数注释
    # 例如：
    gain::Real = 1.0 # 增益

    # Private (Julia 内部变量)
    # 格式：变量名::类型 = 值 # 变量说明
    # 例如：
    _count::Integer = -1 # 计数器

    # Methods (主要调用算法，包括 setupImpl, stepImpl, releaseImpl)

    # 初始化函数：函数名固定，函数形参与 stepImpl 函数形参一致
    function setupImpl(self, u)
        self._count = 0
        # ...
        return nothing
    end #setupImpl

    # 单步计算函数：函数名固定，第一个函数形参必须是 self，其余函数形参将作为 Modelica
    # 组件的输入端口，函数返回值作为输出端口
    function stepImpl(self, u)
        self._count += 1
        # ...
        y = u * self.gain
    end #stepImpl
end
```

```
    return y
end #stepImpl

# 释放资源函数：函数名固定，且只能有一个函数参数 self
function releaseImpl(self)
    # ...
    return nothing
end #releaseImpl

# 其它自定义函数，第一个函数形参数必须是 self
# function xx(self)
#     ...
# end
end
```

编辑 Julia 对象文件时，请注意以下几点

- 请勿删除注释标签，如# Description、# Methods 等。
- 请勿删除初始化函数 setupImpl(), 单步计算函数 stepImpl(), 释放资源函数 releaseImpl()。
- 类名需要具有唯一性，且与文件名一致。

属性

Julia 对象文件中属性分为公共属性 Parameter 和私有属性 Private。属性对应的规则如下表所示。

表 2-15 Syslab Object 属性规则

属性分类	特点	支持的数据类型
Parameter	Sysplorer 中可以直接访问	特定的 Julia 数据类型
Private	只能通过 Syslab Object 的方法来访问，用户不可见	符合 Julia 语法的所有数据类型

可以在 Parameter 参数中使用以下数据类型：

```
String | Bool | Int64 | Integer | Float64 | Real | Number
Vector{Int64} | Vector{Integer} | Vector{Float64} | Vector{Real} | Vector{Number}
Matrix{Int64} | Matrix{Integer} | Matrix{Float64} | Matrix{Real} | Matrix{Number}
```

方法

Julia 对象文件中包含有三个固定方法：

表 2-16 Syslab Object 属性规则

方法	说明
setupImpl	执行设置和初始化任务
stepImpl	系统输出和状态更新方程

releaseImpl	释放使用的资源和执行必要的清理任务
-------------	-------------------

setupImpl

初始化 Syslab Block

A. 语法

```
function setupImpl(self, input1, input2, ...)  
# ...  
return nothing  
end #setupImpl
```

B. 说明

setupImpl 使用一个或多个输入对 Syslab Block 进行初始化和设置，仿真过程中仅运行一次。

C. 示例

此示例说明如何使用 setupImpl 的输入参数 u 初始化内部变量_count。
setupImpl 的返回值必须为 nothing。

```
function setupImpl(self, u)  
self.count = size(u, 2)  
return nothing  
end #setupImpl
```

stepImpl

系统输出和状态更新方程

D. 语法

```
function stepImpl(self, input1, input2, ...)  
# ...  
return output, output2, ...  
end #stepImpl
```

E. 说明

stepImpl 指定运行 Syslab Object 时要执行的算法，运行对象时将会使用输入参数、属性和状态更新方程来计算输出并更新对象的状态值。

F. 示例

此示例说明使用 stepImpl 方法在每一步仿真时将输入参数值均增加 1。

```
function stepImpl(self, x1, x2)
  y1 = x1 + 1
  y1 = x2 + 1
  return y1, y1
end #stepImpl
```

releaseImpl

释放资源

G. 语法

```
function releaseImpl(self)
  # ...
  return nothing
end #releaseImpl
```

H. 说明

releaseImpl 会释放 Syslab Object 使用的资源，例如文件句柄或设备，以及执行任何必要的清理任务。

I. 示例

此示例说明使用 releaseImpl 方法关闭 Syslab Object 打开的文件。

```
function releaseImpl(self)
  fclose(self._fileID);
  return nothing
end #releaseImpl
```

2.2.2.2.2 Syslab Object Block 的开发规范

可以按照下图所示的流程，在系统建模仿真环境中开发 Syslab Object Block。

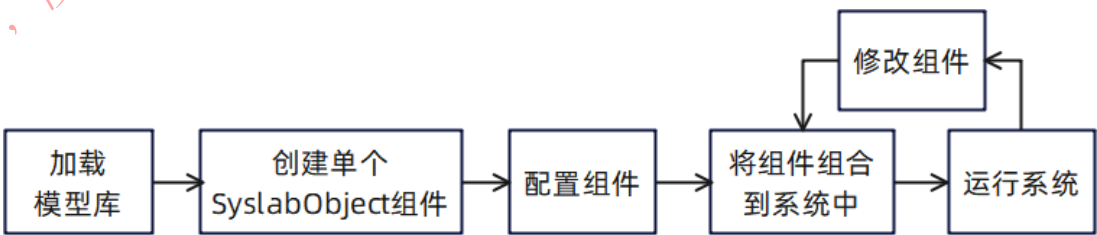


图 2-12 Syslab Object Block 开发流程

- 加载模型库：在 Syslab 中启动 Sysplorer，加载 SyslabWorkspace 模型库
- 创建单个组件：拖拽创建单个 SyslabObject 组件
- 配置组件：关联组件对应的 Julia 对象文件
 - 新建或编辑 SyslabObject 组件的 Julia 对象文件

➤ 编辑 SyslabObject 组件的端口设置

- 将组件组合到系统中：编写一个包含这些 Syslab Object 的 Sysplorer 程序，完成其余部分模型的建模
- 运行系统：在 Sysplorer 中运行您的程序进行仿真
- 修改组件：对已关联 Julia 对象文件的组件进行修改

加载模型库

请通过 Syslab 启动 Sysplorer 软件，确认插件管理中已启用 Syslab Extension for Sysplorer 插件（默认启用）之后，加载 SyslabWorkspace 模型库。

创建单个组件

通过拖拽创建单个 SyslabObject 组件。除了工具箱提供的 Syslab Object 以外，您还可以创建自定义的 Syslab Object，组件在模型库中的路径为 SyslabWorkspace.SyslabObject.SyslabObject。

配置组件

如果您使用某个工具箱中提供的 SyslabObject 组件，可编辑修改默认属性值和方法以便为您的特定系统建立模型。

如果您需要配置自定义 System Object：

1. 点击右键菜单中的“选择 Syslab 对象文件...”项，弹出“Syslab 对象参数”对话框。

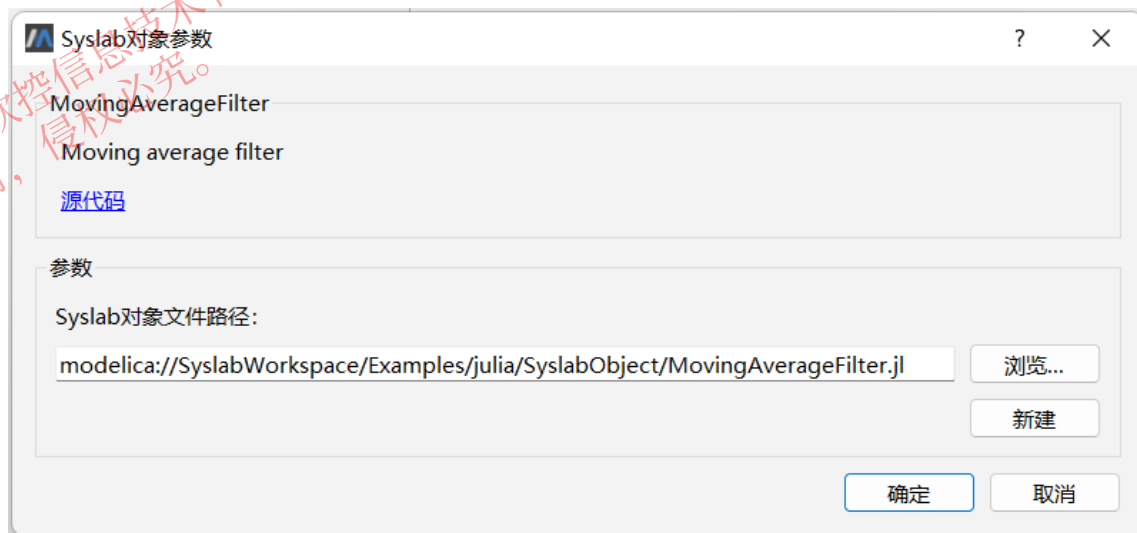


图 2-13 Syslab 对象参数

- 如果您需要新建 Julia 对象文件，点击对话框中“新建”按钮，可以在 Syslab 中打开编辑器并默认新建一个 Julia 对象文件；您也可以通过

Syslab 的“主页”菜单栏中的“新建”下拉框中选择“新建 Julia 对象”进行创建：

- 编辑 Julia 对象文件。
- 保存 Julia 文件，默认类名为 Untitled，请注意文件名需与类名一致，保存后此文件自动关联到 SyslabObject 组件。
- 如果您需要将 SyslabObject 组件关联到现有 Julia 对象文件，点击对话框中的“浏览...”按钮选择对应的 Julia 文件。您也可在文本框中输入文件路径，文件路径支持绝对路径和相对路径。
- 点击对话框中的“确认”按钮，组件依据 Julia 文件自动生成参数、端口等信息。

注意，组件的默认参数、端口信息仅在点击“确认”按钮后进行刷新。如您对 Julia 文件进行了修改，请重新点击“确认”按钮。

2. 点击右键菜单中的“设置 Syslab 对象端口...”项，弹出“设置 Syslab 函数端口”对话框：

- 指定输入/输出端口的类型和维度信息
- 如有需要，可修改输入端口的范围、初值和参数说明的信息

将组件组合到系统中

当您确定了所需组件并已创建和配置 Syslab Object 之后，请组装您的系统。

运行系统

单击 Sysplorer 的“仿真”按钮对系统进行仿真。如果您需要在 Sysplorer 中获取 Syslab Block 的调试信息，请参阅调试信息。

修改组件

如果您需要修改已关联 Julia 文件的 SyslabObject 组件配置，请选中组件后，点击右键菜单中的“选择 Syslab 对象文件...”项，点击弹出对话框中的“源代码”按钮，可在 Syslab 中打开对应文件进行编辑。

完成编辑后，单击对话框中的“确认”按钮，组件将自动更新参数、端口等信息。

对于组件参数，如果已在 Sysplorer 中对类生成的属性初始值进行了修改，比如在 Sysplorer 组件参数底部停靠栏中修改了初始属性值，更新后不会覆盖当前修改值。

对于组件端口，如果您删除了某一端口，更新后对应位置上的原有连接线

将悬空。

您也可以在对话框中新建或更改关联的 **Julia** 对象文件，组件将自动更新。

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。